# CR-SHE: One-round oblivious range analytics over encrypted, replicated IoT data under $(n-1)$-collusion

ARTICLE INFO

*Keywords*:
Range aggregation
Distributed point function
Homomorphic encryption
Internet of Things
Multi-cloud
Oblivious analytics
Summed-area table

ABSTRACT

IoT telemetry replicated across independent clouds must stay encrypted, yet the analytics people run on it are range aggregates: one device's mean temperature over an afternoon, or the hours a pollutant spent in its top bands. Distributed-trust query systems hide which records a query touches, but they compute over plaintext, stop at retrieval, or need an honest majority and server-to-server interaction, while single-server encrypted range aggregation leaks access patterns or pays for homomorphic comparisons. We present CR-SHE, which answers multi-dimensional range aggregates over owner-encrypted, replicated data in one round, hiding the range, the matched records, their number and the result from any coalition of up to $n-1$ non-interacting providers. We first show that this setting requires a homomorphic layer: an additively shared store cannot answer such queries locally. The owner stores encrypted summed-area tables of registered data cubes; a query is $2^d$ distributed-point-function keys with signed outputs at the box's corners, padded with dummy keys so that ranges touching the cube's edge look the same, and each provider returns a single ciphertext. Per-cell MAC keys let the client detect deviating providers, and a bit-packed masked index, 32 times smaller, serves arbitrary selections in a second round. We prove correctness and adaptive security, and evaluate an open implementation on a telemetry-shaped cube of 345,690 cells over 405,184 records: provider cost and the 132 kB response do not change from a one-minute window to the whole timeline, and a query needs 2.7 kB of uplink, against a computed 44 MB for a single server.

## 1. Introduction

The proliferation of connected sensors, wearables and cyber-physical systems has made the Internet of Things (IoT) one of the largest and fastest-growing sources of continuously generated data [1]. Bedside monitors, smart meters, traffic and environmental sensors and industrial controllers emit measurements continuously, and the people who need to query them, among them clinicians, grid operators, city agencies and maintenance contractors, are rarely the ones who own the devices. The natural response is to serve this data from the cloud, and modern deployments increasingly replicate it across several providers. Multi-cloud operation supplies elastic query capacity, improves availability through redundancy across independent infrastructures, and frees data owners from single-vendor lock-in [2, 3]. Yet much of this data is acutely sensitive, including physiological telemetry in connected healthcare, mobility traces in smart cities and diagnostic streams from connected vehicles, so it must be encrypted before leaving the IoT trust domain and stay encrypted in the cloud, even as multiple stakeholders need to query and analyse it.

Encrypting outsourced data with conventional schemes neutralises the reason it was sent to the cloud. A provider holding only ciphertexts can neither locate the records a query needs nor compute over them, forcing either decryption in the cloud, which reintroduces exposure, or a wholesale download to the client, which defeats the purpose of outsourcing. Searchable encryption lets a provider identify records matching an encrypted keyword [4–6], and leveled homomorphic encryption lets it evaluate bounded-depth arithmetic directly on ciphertexts [7, 8]. Both, in their single-server form, leak the access pattern, and a substantial body of leakage-abuse work converts that leakage into plaintext recovery [9–11].

Replicating data across independent providers offers an escape that single-server cryptography cannot. If the client splits its query so that no provider sees it, the access pattern stays hidden as long as one provider abstains from collusion. Splinter [12] established this model with function secret sharing, DORY [13] brought it to encrypted documents, and Waldo [14] extended it to rich predicates over time series. The model is therefore not new, and framing a "cross-cloud collusion adversary" as an unexplored threat would misstate the state of the art. What remains open is narrower and, for IoT analytics, decisive.

*Selection and computation are not the same problem, and hiding the first is worthless if the second gives it back.* A scheme can privately determine which records match a keyword and then, in order to aggregate over them, hand the matched index set to the providers, at which point the coalition learns the access pattern exactly, and across repeated queries the search pattern. Existing distributed-trust systems avoid this by not computing at all (DORY), by computing over data the servers can already read (Splinter, whose databases are public), or by moving to a three-party honest-majority setting with secret-shared data and server-to-server interaction (Waldo). None of them gives analytics over singly-encrypted, owner-keyed data under a $t = n - 1$ collusion bound without inter-provider communication. This paper identifies that composition leak and eliminates it under a constraint set of $t = n - 1$ collusion, no server-to-server channel, and data the providers cannot read. These constraints rule out the obvious alternative of secret-sharing the data, because a shared store cannot answer a hidden query without either server interaction or a single-server private-retrieval primitive (Section 5.2).

*IoT analytics are range queries.* A clinician asks for one patient's mean heart rate across a night, a city agency for the number of hours carbon monoxide sat in its top three bands at one site, a maintenance contractor for a machine's vibration total over a shift. Each is an aggregate over a box in a small number of ordered dimensions: device, time window, value band. Answering such a box with single-keyword aggregates costs one query per tag in the box and reveals its length; retrieving the records first costs a retrieval per tag and hands the client data it did not ask for. Prior distributed-trust systems that support ranges compute over plaintext or need an honest majority with interaction (Section 2), and single-server encrypted range aggregation leaks access patterns or evaluates homomorphic comparisons.

We propose CR-SHE (Collusion-Resilient Searchable Homomorphic Encryption). The data owner replicates a one-time-pad-masked pseudonymous index and a homomorphically encrypted record column to $n$ providers. A query is one distributed point function (DPF) key per provider. Rather than returning the selection vector so the client can name the matched records, each provider contracts its *own share* of that vector against the encrypted column and returns a single ciphertext; the client adds the $n$ ciphertexts and decrypts once. The matched set is never assembled at any provider, its cardinality never leaves the client, and per-query downlink for an aggregate is one ciphertext rather than linear in the record count. The contributions are as follows.

- **One-round oblivious range aggregation** (Section 5.7). The owner registers data cubes over ordered tag families and stores encrypted summed-area tables. A $d$-dimensional box query is $2^d$ DPF keys with signed outputs at its corners, padded with zero-output dummy keys so that ranges touching the cube's edge are indistinguishable, and each provider returns one ciphertext. Provider work and the response are independent of the range and of the record count; per-cell MAC keys verify every corner (Lemma 2); appending to the newest time slice touches only that slice. Section 2 positions this against the closest work, and states what our literature search could and could not reach.

- **End-to-end obliviousness under $t = n-1$ collusion, and why it needs a homomorphic layer.** We give a construction in which the analytics phase inherits the access-pattern hiding of the selection phase, over data encrypted at rest under a single owner's key, with no server-to-server communication. We show that secret-sharing the data instead does not work: local answers over additive shares miss the cross terms, and any scheme that avoids them yields single-server private information retrieval (Proposition 1, Section 5.2). Section 5.3 presents the mechanism, and Remark 3 explains why returning the matched set to the providers admits no simulator, while dealing it as fresh shares does.

- **Oblivious homomorphic aggregation, and a precomputed-column fast path.** We make explicit the compatibility condition (Lemma 1) under which DPF shares and the homomorphic plaintext ring compose without wraparound. The condition is elementary in isolation, but its violation is silent and data-dependent in practice. We then give a variant (Section 5.5) in which single-keyword aggregates over pre-registered templates cost a symmetric full-domain pass plus $O(N/\ell)$ packed ciphertext operations, independent of the record count, and return the value and its MAC for several templates in the two slot rows of one ciphertext (Section 5.6).

- **A flexible path for conjunctions and ad-hoc weights.** Two rounds: a fixed-size retrieval, after which the client forms any selection vector it likes and deals it as fresh additive shares, which each

provider contracts against the encrypted record column (Section 5.10). A coalition sees only uniform shares, so the guarantee is unchanged.

- **A packed, masked index.** A one-time-pad mask, sampled exactly uniformly over $\mathbb{Z}_p$, makes the stored index perfectly hiding under the PRF assumption, and packing 32 membership bits per $\mathbb{Z}_p$ word cuts its size 32-fold and retrieval time 19–27-fold in our measurements, while keeping four bits of headroom to reject corrupted words (Section 5.4, Theorem 1).
- **Detection of inconsistent providers, an authorisation requirement, and streaming ingest.** A MAC'd ciphertext column lets the client reject a response inconsistent with its tag except with probability $1/p \approx 2^{-36}$, which is fault detection rather than integrity. With a single MAC key a consistent linear substitution passes; per-row keys on the fast path catch it unless it touches only the queried row, and we measure both. We show that providers cannot restrict which rows a DPF-hidden query reads, measure the index extraction this permits, and identify the key-carried authorisation proof it requires. An append-only two-tier index supports IoT streams, with forward and backward privacy immediate because the coalition's per-query view is query-independent.
- **Evaluation.** A C++ prototype measured the single-keyword path, the multi-provider deployment and a leakage-abuse attack on two real IoT corpora. An open reference implementation measures range aggregation on a telemetry-shaped cube, the packed index, and a single server holding the same encrypted columns, which isolates what the non-collusion assumption buys (Section 9).

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 fixes notation and primitives. Section 4 presents the system and threat model. Section 5 details the construction, Section 6 the security analysis, and Section 7 the asymptotic costs. Section 8 describes the implementation, Section 9 reports and discusses results, and Section 10 concludes.

## 2. Related work

### 2.1. Distributed-trust query systems

Splinter [12] established the model of splitting a query into function-secret-sharing shares evaluated against replicas, hidden as long as one provider abstains, with SUM, COUNT, MAX and TOPK over *public* databases the providers can read. DORY [13] moves to encrypted documents with a masked Bloom-filter index, hiding access patterns and result counts from all but one server, but stops at retrieval. Waldo [14] supports multi-predicate filtering over time series with malicious security, in a three-party honest-majority setting with secret-shared data and server-to-server interaction, that is, a collusion bound of $t = 1$. CR-SHE trades Waldo's predicate expressiveness and full malicious security for $t = n - 1$, no inter-provider communication and owner-keyed encryption at rest. The primitives are due to Gilboa and Ishai [15] and Boyle, Gilboa and Ishai [16, 17]; FSS also serves metadata-hiding messaging [18], robust private statistics [19, 20], access control for dealers [21] and verifiable keys [22]. At bottom, DPF-based selection is multi-server PIR [23], and its linear server work is the price of PIR without preprocessing [24]; what CR-SHE adds is the ciphertext-side contraction, the masked index, and range aggregation on top of both.

### 2.2. Private range aggregation

Prefix sums answer a range sum from a constant number of lookups, a structure known from data cubes [25] and summed-area tables [26]. Over encrypted data held by one server, range-aggregate schemes built on searchable or structured encryption [27, 28] accept leakage of access patterns or volumes, and HADES [29] evaluates point and range predicates under fully homomorphic encryption in one round, for data that is public. With non-colluding servers, Hayata et al. [30] give FSS-based PIR for range *retrieval*, Hafiz et al. [31] answer aggregate queries in one round with information-theoretic PIR over a database the servers host, and Waldo [14] supports range predicates over secret-shared time series with an honest majority and server interaction. CR-SHE's range path composes prefix-sum cubes with oblivious homomorphic aggregation, so that the data stay encrypted under the owner's key, the coalition bound is $n - 1$, and providers do not interact. Searches of web indexes and of the Consensus paper index found no earlier description of that

**Table 1**
CR-SHE against distributed-trust query systems and encrypted-search baselines. *Oblivious analytics* means the servers learn neither which records feed the aggregate nor how many.

| System | Storage at the servers | Search / filter | Oblivious analytics | Range aggregates | Collusion bound | Server channel | Malicious security | Hides $\|S\|$ |
|---|---|---|---|---|---|---|---|---|
| Splinter [12] | plaintext replica | ✓ | ○ (plaintext) | ? | $n-1$ | no | × | pad to $k$ |
| DORY [13] | encrypted replica | ✓ | × | × | $n-1$ | no | detect | ✓ |
| Waldo [14] | secret-shared | ✓ | ✓ | ✓ | 1 (h. maj.) | yes | ✓ | ✓ |
| Hafiz et al. [31] | hosted database | ✓ | ✓ | ? | ? | no | ? | ? |
| Timecrypt, Zeph [50, 51] | encrypted, 1 server | time only | ✓ | time, revealed | n/a | n/a | × | × |
| HADES [29] | public, 1 server | ✓ | ✓ | ✓ | n/a | n/a | ? | ? |
| ARQ [28] | encrypted, 1 server | range | × | ✓ (leaky) | n/a | n/a | × | × |
| Multi-cloud SSE [43] | encrypted | ✓ | × | × | × | no | × | × |
| $(n,n)$-shared, local (§5.2) | secret-shared | ✓ | ×‡ | n/a | $n-1$ | no | n/a | n/a |
| Single server, enc. selection | encrypted, 1 server | ✓ | ✓ | ✓ | none needed | n/a | × | ✓ |
| **CR-SHE (this work)** | encrypted replica | ✓ | ✓ | ✓, 1 round | $n-1$ | no | detect | ✓ / pad† |

✓: supported; ○: partial; ×: not supported; ?: not determined from the sources we could read (abstracts or search summaries only). *Detect* denotes detection of a deviating server without guaranteed output delivery. †Aggregates hide $|S|$; retrieval pads to the public bound $k_{\max}$. ‡Local answers over additive shares miss the cross terms (Proposition 1); fixing that needs server interaction or a single-server PIR per share. The single-server row is baseline (vi): no non-collusion assumption, but an encrypted query linear in the domain (for a range query on our cube, about 44 MB against 2.7 kB of DPF keys).

combination, but the full texts of [28–31] could not be opened from our environment, so we make no claim of priority; the search log ships with our artefact.

### 2.3. Private information retrieval

PIR with preprocessing breaks the linear lower bound: offline hints let the online phase run in sublinear time, with two servers [32] or one [33, 34]. Those hints are client state computed over the whole database and invalidated by updates, which suits a stable table read by one client better than a stream ingested continuously and queried by many stateless data users; we keep the linear pass and list preprocessing as future work. Single-server PIR from lattice-based encryption [35–38] removes the non-collusion assumption altogether [39]. For the fast path in isolation it is a genuine alternative, being a PIR query into a table of owner-encrypted aggregates, and we measure it directly as baseline (vi) (Section 5.6): on that path the trust split saves a constant factor in server work and three to four orders of magnitude in uplink, not asymptotic work. On retrieval and the flexible path a single server must multiply an encrypted selection vector by the whole $N \times N_d$ index, at least $N\lceil N_d/32\rceil/\ell$ homomorphic products with the same bit packing, which we bound at 1.5–3.1 times CR-SHE's retrieval in the same implementation; again a constant factor, with the uplink gap on top.

### 2.4. Encrypted search, encrypted analytics and multi-cloud storage

Searchable encryption leaks search and access patterns, which leakage-abuse attacks turn into plaintext recovery [9–11]; mitigations include ORAM [40], volume-hiding structured encryption [41, 42], multi-cloud variants [43, 44] and richer or committee-authorised search [45–47]. Leveled homomorphic encryption [7, 8] underpins encrypted IoT analytics [48, 49]; Timecrypt [50] and Zeph [51] aggregate encrypted time series but reveal the queried interval. IoT search has been studied for sensing platforms [52] and vehicles [53], and multi-owner analytics with MPC and HE [54]. Multi-cloud storage designs disperse trust with hybrid encryption, secret sharing or attribute-based control [55, 56], or compute across clouds with partial HE [57]; systematisations note the lack of standard trust models [58]; and encrypted databases [59–61] rely on a trusted proxy or hardware rather than on non-colluding providers.

## 3. Preliminaries

This section fixes notation (Table 2) and recalls the primitives CR-SHE combines: a pseudorandom function for the index mask, additive secret sharing, distributed point functions for private selection across the replicated index, and leveled homomorphic encryption for aggregation on ciphertexts.

**Table 2**
Notation.

| Symbol | Meaning |
|---|---|
| $\lambda$ | security parameter |
| $n,\ t$ | number of providers; collusion bound ($t = n-1$) |
| $N,\ N_d$ | index (tag) domain size; number of records |
| $p$ | HE plaintext modulus; also the DPF output group |
| $\ell$ | number of SIMD slots per ciphertext |
| $Q$ | number of adaptive queries |
| $D$ | secret, append-only tag dictionary, $w \mapsto \alpha_w$ |
| $F',\ K'$ | PRF for the index mask; its key, fresh per static tier |
| $I,\ I'$ | membership index in $\{0,1\}^{N\times N_d}$; masked index |
| $P$ | one-time-pad mask, rejection-sampled from $F'_{K'}$, uniform on $\mathbb{Z}_p$ |
| $\alpha_w$ | row of keyword $w$, $\alpha_w = D(w)$ |
| $b$ | record selection vector, $b = I[\alpha_w] \in \{0,1\}^{N_d}$ |
| $b_j$ | provider $\mathsf{CP}_j$'s additive share of $b + P[\alpha_w]$ |
| $(\mathsf{Gen}, \mathsf{Eval})$ | distributed point function (DPF) |
| $\kappa_j$ | DPF key sent to provider $\mathsf{CP}_j$ |
| $e_j$ | full-domain evaluation vector, $e_j[x] = \mathsf{Eval}(j, \kappa_j, x)$ |
| $pk,\ sk$ | HE public / secret key |
| $m_i,\ \mathsf{CT}_i$ | computable field of record $i$; its ciphertext |
| $\delta,\ \widehat{\mathsf{CT}}_i$ | MAC scalar; MAC column $\mathsf{Enc}(\delta m_i)$, in slot row 1 |
| $\delta_x,\ K''$ | per-row (per-cell) MAC key $F''_{K''}(x)$; its key |
| $f,\ w_f$ | aggregate template; its public weight vector |
| $A_f,\ \widehat{A}_f$ | precomputed aggregate column; its MAC column |
| $v,\ v_j$ | flexible selection vector over records; $\mathsf{CP}_j$'s share |
| $d,\ M$ | cube dimension; number of cells $\prod_k s_k$ |
| $C_f,\ T_f$ | cell aggregates; their summed-area table |
| $\kappa_\sigma,\ \beta_\sigma$ | corner $\sigma \in \{0,1\}^d$ of a box; its sign or 0 |
| $S$ | matched record set; $\lvert S\rvert$ its cardinality |
| $\mathcal{L}$ | leakage profile |
| $C$ | corrupted provider coalition ($\lvert C\rvert \le t$) |
| $\mathrm{DO},\ \mathrm{DU},\ \mathsf{CP}_j$ | data owner; data user; $j$-th cloud provider |

*Pseudorandom functions.* $F : \{0,1\}^\lambda \times \{0,1\}^* \to \{0,1\}^\lambda$ is a PRF if for every PPT distinguisher $\mathcal{B}$, $\mathbf{Adv}^{\mathrm{prf}}_{F,\mathcal{B}}(\lambda) = |\Pr[\mathcal{B}^{F_k(\cdot)} = 1] - \Pr[\mathcal{B}^{R(\cdot)} = 1]| \le \mathsf{negl}(\lambda)$ for a uniformly random key $k$ and a truly random function $R$ [62].

*Additive secret sharing.* An $(n,n)$ additive sharing splits $m \in \mathbb{Z}_p$ into $(m_1, \ldots, m_n)$ with $\sum_j m_j = m$; any $n-1$ shares are uniform and independent of $m$.

*Distributed point functions.* An $n$-party DPF [15–17] for point functions $f_{\alpha,\beta} : [N] \to \mathbb{G}$ is a pair $(\mathsf{Gen}, \mathsf{Eval})$ with *correctness* $\sum_{j\in[n]} \mathsf{Eval}(j, \kappa_j, x) = f_{\alpha,\beta}(x)$ for all $x$, and $(n-1)$-*privacy*: a PPT simulator $\mathcal{S}_{\mathrm{DPF}}$ exists such that for every $C \subseteq [n]$ with $|C| \le n-1$, $\{\kappa_j\}_{j\in C} \approx_c \mathcal{S}_{\mathrm{DPF}}(1^\lambda, C, N)$. The two-party instantiation has keys of size $O(\lambda \log N)$ and admits full-domain evaluation in $O(N)$ pseudorandom-generator calls through a single traversal of the GGM tree, rather than the $O(N \log N)$ that $N$ independent $\mathsf{Eval}$ calls would cost. For $n > 2$, succinct $(n-1)$-private constructions exist whose key size grows with both $n$ and $\sqrt{N}$ [16]. Our prototype does not implement one: it instantiates $n > 2$ with a simpler sharing in which $n-1$ providers hold a $\lambda$-bit seed whose counter-mode expansion is their share, and one provider holds the explicit difference vector. That construction is correct by inspection and $(n-1)$-private under the PRF assumption: any coalition of $n-1$ providers is missing at least one seed expansion, which would hide $\alpha$ perfectly were it truly random,

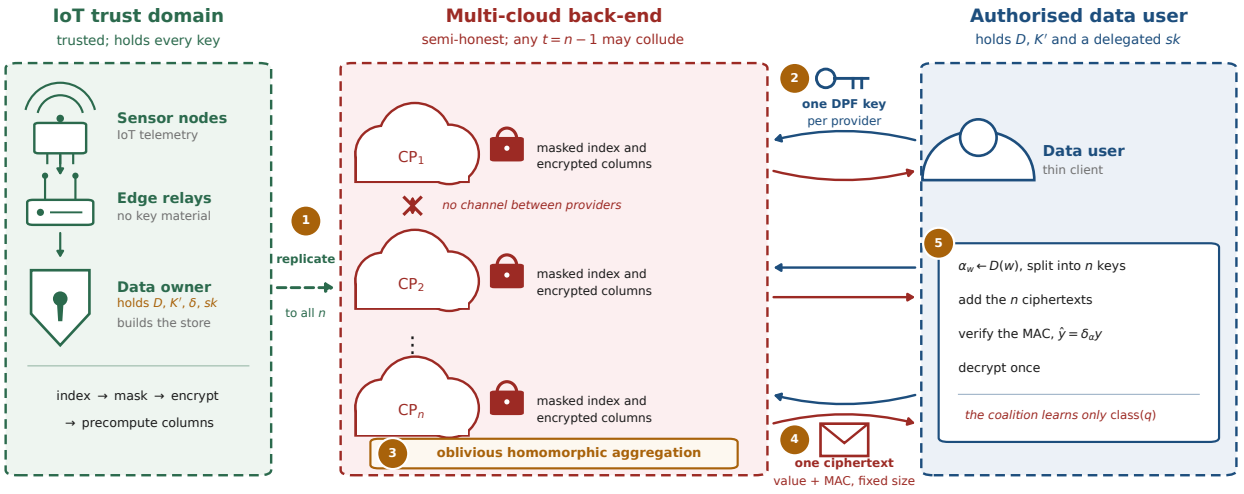



**Figure 1:** Overview of CR-SHE. The owner replicates a masked index and encrypted columns to $n$ providers (1). A query is split into one DPF key per provider (2), each provider aggregates obliviously over its own share (3) and returns one ciphertext carrying value and MAC (4), and the user combines, verifies and decrypts (5). No provider ever sees the matched set or its size.

and is a PRF output in fact. Its per-provider key is $\lambda$ bits for $n-1$ providers and $N\lceil \log p \rceil$ bits for one. This is not function secret sharing in the succinct sense, being the additive sharing of a unit vector familiar from multi-server PIR, so our key-size claims are for $n = 2$, and we use $n \in \{3, 4\}$ only to show that provider compute is flat in $n$; the uplink it costs is reported separately (Section 9.10).

*Leveled homomorphic encryption.* $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec})$ is correct for arithmetic circuits of multiplicative depth $\leq L_{\mathrm{HE}}$ over plaintext ring $\mathbb{Z}_p$ and IND-CPA secure with advantage $\mathbf{Adv}^{\mathrm{cpa}}$ [7, 8]. We write $\boxplus, \boxminus$ for ciphertext addition and subtraction and $\boxdot$ for plaintext-to-ciphertext multiplication, and use SIMD packing with $\ell$ slots. Providers hold only $pk$; $sk$ never leaves the DO or DU. Opaque payloads use an IND-CPA symmetric scheme.

## 4. System and threat model

The **data owner** (DO), an IoT-domain authority such as a hospital, fleet operator or city data office, holds the tag dictionary $D$, the mask key $K'$, the MAC keys and the HE secret key $sk$, and keeps the plaintext records in cold storage to precompute columns and rebuild tiers. **Producers** and **edge relays** generate and forward records and hold no secrets. The $n$ **cloud providers** $\mathrm{CP}_1, \ldots, \mathrm{CP}_n$ each hold $pk$ and a replica of the masked index and the encrypted columns and tables, and never decrypt. Authorised **data users** (DU) hold $D$, $K'$ and delegated decryption, issue queries, and combine the providers' responses (Fig. 1). For example, a city's monitoring stations feed the data office, which replicates the encrypted store to three clouds; analysts at a health agency ask for the hours carbon monoxide spent in its top bands at one site last month, and no cloud learns the site, the month, the bands, the number of matching hours or the answer.

The adversary is probabilistic polynomial-time, static and semi-honest: it fixes a coalition $C \subseteq [n]$ with $|C| \leq t = n - 1$, whose members follow the protocol and pool everything they see, namely

$$\mathrm{View}_C = \big(I',\ \mathrm{CT},\ \widehat{\mathrm{CT}},\ pk,\ pp,\ \{A_f, \widehat{A}_f\}_f,\ \{T_f, \widehat{T}_f\}_{f,\mathrm{cube}},\ \{\kappa_{q,j}, v_{q,j}\}_{j\in C,\, q\in[Q]}\big),$$

plus the responses they compute, over adaptive queries. The goal is that this view be simulatable from data-independent leakage revealing neither the keyword or range, the access pattern, the result-set size, nor the result. The DO and authorised DUs are honest, channels are authenticated and confidential, and side

channels beyond the message sizes accounted for here are out of scope. Malicious providers are handled only to the extent of the MAC checks of Section 5.8; collusion between a DU and a provider is excluded, since a DU already holds decryption capability, but a DU exceeding its authority is discussed in Section 5.9; and a compromised DU exposes all data, since decryption is delegated in full.

# 5. The CR-SHE construction

## 5.1. Overview

The DO builds a membership index $I \in \{0,1\}^{N \times N_d}$ in which keyword $w$ occupies row $\alpha_w$ and $I[\alpha_w][i] = 1$ if and only if record $i$ contains $w$. Rows are assigned by a secret, append-only dictionary $D$: a keyword receives the next free row when it first appears and keeps it for the lifetime of the deployment, so rows never move when keywords are added (Section 5.11). The DO shares $D$ with authorised DUs and keeps it from the providers, because the vocabulary itself, being device and site names, is sensitive. Which row a keyword gets carries no security weight, since providers see neither addresses nor unmasked rows; only injectivity matters, for correctness. Rows are fixed width, so no row is distinguishable by size, and the index is stored masked as $I'[x][i] = I[x][i] + P[x][i] \bmod p$ with $P$ drawn from the PRF $F'_{K'}$ by rejection sampling, so that every entry is uniform on $\mathbb{Z}_p$ (Section 5.4). The computable field of each record is encrypted as $\mathrm{CT}_i = \mathsf{Enc}_{pk}(m_i)$, with a MAC column $\widehat{\mathrm{CT}}_i = \mathsf{Enc}_{pk}(\delta m_i)$; opaque payloads are encrypted symmetrically.

A design that first recovers the selection vector at the client and then asks the providers to evaluate $f$ over $\{\mathrm{CT}_i\}_{i \in S}$ destroys in the second step exactly what the first protected, because the providers are handed $S$. CR-SHE therefore never materialises $S$ at a provider. Each provider contracts its own share of the selection vector against the encrypted column and returns one ciphertext.

## 5.2. Why a homomorphic layer is necessary

The obvious alternative to replicating ciphertexts is to deal the data to the providers as an $(n, n)$ additive sharing over $\mathbb{Z}_p$ and answer DPF queries over the shares, with no homomorphic encryption at all. Under our constraints it does not work, and the reason fixes what any design here must contain.

**Proposition 1 (No local answers over shares).** *Let a column $A \in \mathbb{Z}_p^N$ be dealt as $A = \sum_{k \in [n]} S_k$, with $S_1, \ldots, S_{n-1}$ uniform and $CP_k$ holding $S_k$, and let $e_j$ be the evaluation vector of DPF key $\kappa_j$ for $f_{\alpha,1}$. If each provider returns $r_j = \langle e_j, S_j \rangle$, then*

$$\sum_j r_j - A[\alpha] \;=\; -\sum_{j \neq k} \langle e_j, S_k \rangle,$$

*and for $n = 2$ this error is uniform on $\mathbb{Z}_p$ over the choice of $S_1$, so the answer is correct with probability $1/p$.*

Proof. $A[\alpha] = \langle \sum_j e_j, \sum_k S_k \rangle = \sum_{j,k} \langle e_j, S_k \rangle$; the providers' answers supply only the diagonal. For $n = 2$, substituting $S_2 = A - S_1$ gives the error $-\langle e_1, A \rangle + \langle e_1 - e_2, S_1 \rangle$, which is uniform because $S_1$ is uniform and independent of the keys, provided $e_1 \neq e_2$. Equality would mean $e_1 = e_2 = 2^{-1} u_\alpha$, which reveals $\alpha$ to either provider and so cannot occur for a private DPF. □

The cross terms are not an artefact of choosing linear answers. Take any non-interactive scheme in which $\mathrm{CP}_1$ holds an additive share $S_1$ of the data, the client forms its queries without reference to the data, providers answer independently, and every query and response is shorter than a share. Given a database $X$ held by a single server, a client can run that scheme with $S_1 := X$ and the other shares drawn by itself, which fixes the data as $A = X + \sum_{k>1} S_k$. It plays providers $2, \ldots, n$ locally, sends only the first query to the server, and subtracts $\sum_{k>1} S_k[\alpha]$ from the result. Privacy against the single provider $\mathrm{CP}_1$ hides $\alpha$ from the server, so this is a non-trivial single-server PIR scheme for $X$, and such schemes imply oblivious transfer [63]; their standard instantiation is homomorphic encryption [39]. Secret-sharing the data therefore does not remove the public-key primitive, it only moves it into a per-share PIR; the alternative is to let providers interact, for example through multiplication triples, which the no-channel constraint excludes. A store replicated under one owner's homomorphic key is the direct way to supply the primitive, and it also brings three operational properties that a shared store lacks.

*Durability.* A sharing with privacy threshold $n-1$ is lost when any one share is lost, whereas a replica set survives the loss of $n-1$ providers, which is the reason multi-cloud deployments replicate at all. Threshold sharing does not escape the trade-off: a $(t{+}1, n)$ scheme tolerates $n-t-1$ losses, so at $t = n-1$ it degenerates to the $(n, n)$ case.

*Graceful degradation.* A query needs every provider in the set it is addressed to. With replicas, queries continue over any surviving set of at least two providers, at that set's collusion bound, without touching stored state; a shared store cannot answer until the dealer re-shares.

*Key management and churn.* Encryption under one owner key lets the owner rotate keys, revoke a provider, or add one by copying ciphertexts. Under sharing, every change to the provider set re-deals the whole store, and the dealer must hold the plaintext to do it.

### 5.3. Oblivious homomorphic aggregation

Suppose the providers hold additive shares $v_1, \ldots, v_n \in \mathbb{Z}_p^M$ of a vector $v$ and replicas of an encrypted column $\mathrm{C} = (\mathsf{Enc}_{pk}(c_1), \ldots, \mathsf{Enc}_{pk}(c_M))$. Provider $j$ returns

$$\mathrm{ct}_j \;=\; \boxplus_{i\in[M]} v_j[i] \boxdot \mathrm{C}_i,$$

and the DU computes $\boxplus_j \mathrm{ct}_j$, which decrypts to $\langle v, c\rangle$, and decrypts once. Nothing that leaves a provider depends on $v$ beyond its own share, and the response is one ciphertext of fixed size whatever the support of $v$. CR-SHE instantiates this contraction twice. On the fast path (Section 5.5) $v$ is the DPF's unit vector $u_{\alpha_w}$, its shares are the evaluation vectors $e_j$, and C is a precomputed aggregate column over the tag domain. On the flexible path (Section 5.10) $v$ is a selection vector over the records that the client forms after a retrieval and deals as fresh shares, and C is the record column. In both cases the matched set is never materialised at a provider.

**Lemma 1 (Algebraic compatibility).** *If the DPF output group and the HE plaintext ring are the same $\mathbb{Z}_p$, and $\sum_i |w_f[i]\, m_i| < p$ for every admissible template, then for shares $b_1, \ldots, b_n \in \mathbb{Z}_p^{N_d}$ with $\sum_j b_j = \beta \bmod p$,*

$$\mathsf{Dec}_{sk}\Big( \boxplus_{j\in[n]} \; \boxplus_{i\in[N_d]} \; b_j[i] \boxdot CT_i \Big) \;=\; \sum_{i\in[N_d]} \beta[i]\, m_i \pmod p.$$

Proof. $\boxdot$ and $\boxplus$ are homomorphisms over the same ring $\mathbb{Z}_p$, so summation commutes with decryption and no reduction is lost: share addition and plaintext addition are taken modulo the same modulus. The magnitude bound prevents plaintext overflow. What remains is decryption correctness, which is a noise-budget question rather than a depth question. A plaintext-to-ciphertext product consumes no multiplicative level but scales the noise by up to the norm of the plaintext, here as large as $p \approx 2^{36}$, because DPF shares are uniform in $\mathbb{Z}_p$, and the contraction sums $\lceil N_d/\ell \rceil$ such products before a rotate-and-sum. The parameters must leave room for that growth. We check the worst case, full-magnitude coefficients across several packed chunks, empirically rather than assume it (Section 9.1). □

Lemma 1 is what makes the two primitives compose. Instantiating the DPF over $\mathbb{Z}_{2^k}$ and the homomorphic scheme over an unrelated plaintext modulus, which is the natural default since the two libraries are chosen independently, silently breaks correctness through wraparound, and the failure is data-dependent and therefore easy to miss in testing.

### 5.4. Masking the index

Retrieval (Section 5.9) reads a row of the membership index through the DPF. Stored in the clear, that index would hand every provider each posting-list length, which the attack of Section 9.11 turns into tag identities. The DO therefore stores it masked and packed:

$$I'[x][c] \;=\; \sum_{b<\pi} I[x][\pi c + b]\, 2^b \;+\; P[x][c] \bmod p,$$

with $\pi = 32$ membership bits per word, where row $P[x]$ is the output of $F'_{K'}$ on $(x, \cdot)$ turned into exactly uniform elements of $\mathbb{Z}_p$ by rejection sampling: each 64-bit PRF word is cut to $\lceil \log_2 p \rceil = 37$ bits and kept only if it is below $p$, which accepts a candidate with probability $p/2^{37} \approx 0.5$. A packed word is below $2^{32} < p$, so the pad still hides it perfectly, and the index occupies $\lceil N_d/32 \rceil$ words per row instead of $N_d$: a 32-fold saving in storage, in the retrieval pass and in its response. Because $p$ exceeds $2^{36}$, the four spare bits double as a sanity check: a word at or above $2^{32}$ after unmasking can only come from a corrupted share, and a uniformly corrupted word passes with probability 1/16. A retrieval returns shares of $I'[\alpha_w] = \mathrm{pack}(b) + P[\alpha_w]$, and the DU, which derives $P[\alpha_w]$ from $K'$, subtracts it and unpacks. No provider ever sees an unmasked row or a mask value, and no correction ciphertext is needed.

**Theorem 1 (Index hiding).** *For any two membership indexes $I_0, I_1 \in \{0,1\}^{N \times N_d}$, no* PPT *distinguisher tells $I_0 + P$ from $I_1 + P$ with advantage greater than $\mathbf{Adv}^{\mathrm{prf}}_{F'}$. In particular the coalition learns no posting-list length, no keyword-frequency multiset, and no membership relation from the stored index.*

Proof. Replace $F'$ by a truly random function at the cost of $\mathbf{Adv}^{\mathrm{prf}}$. A candidate is then uniform on $\{0, \ldots, 2^{37}-1\}$, and conditioning on acceptance makes each mask value exactly uniform on $\mathbb{Z}_p$, independently of every other. $I_b + P$ is then a one-time pad of $I_b$ and is uniform on $\mathbb{Z}_p^{N \times N_d}$ for either $b$. □

Reducing 64-bit words modulo $p$ instead, as is common, would carry a statistical term $NN_d\, 2^{-64} \approx 2^{-31.9}$ on the full telemetry index; rejection sampling removes it at the cost of about two PRF words per mask value.

**Remark 1 (Stripping the mask first).** An alternative contracts the masked shares $b_j$ directly against the record column and cancels the resulting term $u_f(\alpha_w) = \sum_i w_f[i]\, P[\alpha_w][i]\, m_i$ with an encrypted correction column $U_f = (\mathsf{Enc}_{pk}(u_f(x)))_{x \in [N]}$, retrieved by the same DPF. An earlier version of our design did this. It has two defects. $U_f$ must be precomputed by the DO for each weight vector, so it serves only registered weights, for which the fast path is cheaper; it cannot serve the ad-hoc weights and conjunctions it was meant for. And the correction must itself be encrypted: published in the clear, the count correction $g(x) = \sum_i P[x][i]$ gives $\sum_i I'[x][i] - g(x) = |I[x]|$, the row Hamming weight, restoring the volume leakage the mask removes. The flexible path avoids both by stripping the mask at the client before any contraction.

### 5.5. Precomputed columns: the single-keyword fast path

The contraction of Section 5.3 costs $\Theta(NN_d)$ ring operations per provider per query, which is the price of hiding the access pattern without preprocessing, by the lower bounds underlying ORAM and PIR [24, 40]. For the common case, an aggregate over the whole posting list of a single keyword under a template registered at setup, this cost is avoidable. The DO precomputes, for every $x \in [N]$,

$$A_f[x] \;=\; \sum_{i \in [N_d]} I[x][i]\, w_f[i]\, m_i \bmod p,$$

and publishes it as $\lceil N/\ell \rceil$ packed ciphertexts. A query then reduces to an oblivious retrieval of one element of an encrypted length-$N$ array:

$$\mathrm{ct}_j \;=\; \boxplus_{x \in [N]} e_j[x] \boxdot A_f[x],$$

costing $O(N)$ pseudorandom-generator calls for the full-domain evaluation plus $O(N/\ell)$ packed ciphertext-plaintext multiplications and $O(\log \ell)$ rotations, *independent of* $N_d$. No mask correction is needed on this path, because $A_f$ is derived from the unmasked $I$ held by the DO and is itself encrypted.

The fast path supports the templates registered at setup, namely count, sum, per-sensor-band sums, and fixed plaintext-weighted inner products, applied to a single keyword. Conjunctive queries and ad-hoc weight vectors chosen at query time use the flexible path of Section 5.10. Insertion is cheap: a new record carrying $\tau$ tags updates $A_f$ at $\tau$ addresses in plaintext at the DO, followed by re-encryption of all of the column's packed ciphertexts, for the reason given in Section 5.11; no decryption is involved.

### 5.6. Packing the response

The straightforward layout spends two ciphertexts on one scalar: the value and its MAC (Section 5.8), each column packed across all $\ell$ slots and summed over all of them, so that $\ell - 1$ slots of each response go unused. BFV slots have more structure than that. With ring dimension $\ell$ they form two rows of $\ell/2$, and the automorphisms behind rotation act within each row [64]. CR-SHE uses that shape directly. The DO packs $A_f$ in row 0 and $\widehat{A}_f$ in row 1, chunked $\ell/2$ tags per ciphertext; the provider multiplies each chunk by its evaluation vector, replicated in both rows, adds the chunks, and rotates and sums within the rows. Slot 0 then carries the value share and slot $\ell/2$ the MAC share, in one ciphertext.

Several templates share that ciphertext. For $k$ templates, $k$ a power of two, each row is cut into $k$ segments of $s = \ell/(2k)$ slots, template $r$ occupying segment $r$ of both rows, with $\lceil N/s \rceil$ chunks. Rotating and summing by $1, 2, \ldots, s/2$ leaves the sum of segment $r$ in its first slot, so slot $rs$ of each row carries template $r$'s value or MAC share. The response is one ciphertext for any $k \le \ell/2$; what grows with $k$ is the number of chunks, $\lceil kN/(\ell/2) \rceil$ plaintext-ciphertext products, while the rotations fall from $\log_2(\ell/2)$ to $\log_2 s$. The worked instance of a MEAN below needs its count and sum in one response; count, sum and sum of squares give a variance in one round.

Finally, the provider drops the response to a single RNS tower by modulus switching before it sends it. At multiplicative depth one, decryption is unaffected, and the ciphertext halves. Against the two-ciphertext layout, which costs $2\lceil N/\ell \rceil$ products and $2\log_2 \ell$ rotations per template, packing halves the provider's homomorphic work (rotations dominate at these parameters) and quarters the downlink, and for four templates cuts the downlink sixteen-fold (Section 9.6). The natural single-server alternative, one server answering the same query over the same encrypted columns, uses this same layout with an encrypted one-hot query in place of the DPF shares; Section 9.6 measures it.

### 5.7. Oblivious range aggregation over data cubes

A single tag answers a narrow question: the mean over one minute, one site, or one value band. The questions IoT analysts ask are ranges, such as the mean temperature at one device over an afternoon, or the number of hours in which a pollutant sat in its top three bands. Composing such a query from single-tag aggregates costs one query per tag in the range and reveals its length, and the flexible path costs a retrieval per tag. CR-SHE answers rectangular ranges in one round, at a cost independent of both the range and the record count.

*Data cubes.* The DO registers a *data cube*: $d$ ordered tag families $\mathcal{F}_1, \ldots, \mathcal{F}_d$ of sizes $s_1, \ldots, s_d$, for example device, one-minute window and the decile band of one sensor, such that every record $i$ falls in exactly one cell $c(i) \in \prod_k [s_k]$. For each registered template $f$ the cell aggregate is $C_f[c] = \sum_{i:\, c(i)=c} w_f[i]\, m_i$, and the DO stores the $d$-dimensional prefix sums, the summed-area table

$$T_f[c] \;=\; \sum_{c' \le c} C_f[c'],$$

with $\le$ taken coordinate-wise, flattened row-major into a column of $M = \prod_k s_k$ entries and packed like a fast-path column (Section 5.6). Prefix-sum cubes are a classical OLAP structure; what is new here is answering them obliviously over encrypted replicas.

*Queries.* For a box $B = \prod_k [lo_k, hi_k]$ and $\sigma \in \{0,1\}^d$, let corner $\kappa_\sigma(B)$ take $lo_k - 1$ on the axes where $\sigma_k = 1$ and $hi_k$ elsewhere. By inclusion–exclusion,

$$\sum_{c \in B} C_f[c] \;=\; \sum_{\sigma \in \{0,1\}^d} (-1)^{|\sigma|}\, T_f[\kappa_\sigma(B)], \tag{1}$$

where a corner with a coordinate equal to $-1$ lies outside the cube and contributes 0. The DU sends, for every $\sigma$, one DPF key per provider for the point function at $\kappa_\sigma(B)$ with output $\beta_\sigma = (-1)^{|\sigma|} \bmod p$. For a corner outside the cube it sends a *dummy* key, with $\beta_\sigma = 0$ at a uniformly random point. DPF keys hide $\beta$ as well as the point, so every query to a cube carries exactly $2^d$ indistinguishable keys per provider, and a provider cannot tell a range that starts at the cube's edge from one in its interior.

*Answering.* Provider $j$ evaluates its $2^d$ keys over the full domain of $M$ cells. In the *per-corner* layout the encrypted table occupies $2^d$ slot segments per template, and segment $\sigma$ is contracted against $e_{\sigma,j}$; in

the *combined* layout the provider adds $e_j = \sum_\sigma e_{\sigma,j}$ first and contracts once. Either way the response is one ciphertext. The DU adds the $n$ responses and decrypts. In the combined layout slot 0 holds the range aggregate directly. In the per-corner layout it holds each signed corner value $\beta_\sigma T_f[\kappa_\sigma]$, which the DU checks against a per-cell MAC key (row 1 of the segment holds $\delta_c T_f[c]$, with $\delta_c = F''_{K''}(\mathsf{cube}, c)$ domain-separated from the fast path's per-row keys); a dummy corner must decrypt to 0 in both rows. The DU adds the corners. In the combined layout row 1 holds $\delta T_f[c]$ under the global key, so slot $\ell/2$ carries $\delta$ times the aggregate.

*Cost.* Per provider: $2^d$ full-domain DPF evaluations over $M$ cells, $\lceil kM/(\ell/2)\rceil$ plaintext-ciphertext products with $k = 2^d|\{f\}|$ segments (per-corner) or $k = |\{f\}|$ (combined), and $\log_2(\ell/2k)$ rotations. Uplink is $2^d$ two-party keys of $O(\lambda \log M)$ bits each. Nothing depends on the box or on $N_d$. The per-corner layout stores each table $2^d$ times in exchange for per-corner verification (Lemma 2).

*Magnitude and updates.* Every table entry is at most the cube's total, so Lemma 1 requires $\sum_i w_f[i]m_i < p$ over the whole corpus, a stronger condition than for a single tag. With time as the first axis, a batch of records appended to the newest time slice changes $T_f$ only in that slice, and the DO re-encrypts only the ciphertexts covering it; since time advances publicly, that reveals nothing new.

*Why prefix sums rather than interval FSS.* Distributed comparison functions [17] share one-dimensional interval indicators, and in one dimension contracting such shares against the cell column is an alternative to prefix sums. A $d$-dimensional box indicator is the product of $d$ interval indicators, which providers cannot form from shares of the factors without interacting; function secret sharing for boxes as a function class in its own right is a possible alternative that we have neither implemented nor compared. Prefix sums turn the box into $2^d$ points, so the point-function DPF and the homomorphic contraction are used unchanged.

## 5.8. Verification of the aggregate

Providers run the same contraction against the MAC column $\widehat{\mathrm{CT}}$ (or $\widehat{A}_f$ on the fast path), which sits in the second slot row of the same ciphertext (Section 5.6), so $\widehat{\mathrm{ct}}_j$ travels with $\mathrm{ct}_j$ at no extra downlink. The DU decrypts once and aborts unless $\hat{y} = \delta y$. The check is linear, and that fixes precisely what it covers. A provider that perturbs its response by anything it cannot also MAC, such as adding a constant, returning a stale or foreign ciphertext, or corrupting one of the two halves, is detected except with probability $1/p \approx 2^{-36}$ over the unknown $\delta$, which is adequate for catching faults but is not a cryptographic integrity guarantee. Two independent MAC columns would lower this to $1/p^2 \approx 2^{-72}$, sharing the response through a further segment. A provider that applies the *same* linear substitution to both halves passes: replacing its evaluation vector $e_j$ by any $e'_j$, for instance $k\,e_j$, shifts $y$ by $\sum_x (e'_j - e_j)[x]\,A_f[x]$ and $\hat{y}$ by $\delta$ times as much, so the tag still verifies. The provider cannot aim the shift, since it knows neither $A_f$ nor $\alpha_w$, but it can make the output wrong undetectably. The MAC therefore protects against faulty and non-adaptive corruption, not against a provider that deviates on purpose; per-row keys (Remark 2) narrow the gap on the fast path, and Section 9.14 measures all of these cases.

**Remark 2 (Per-row MAC keys).** Replacing the single $\delta$ by per-row keys $\delta_x = F''_{K''}(x)$, storing $\widehat{A}_f[x] = \delta_x A_f[x]$ and checking $\hat{y} = \delta_{\alpha_w} y$, costs no additional ciphertexts or operations. A substitution $e'_j = e_j + d$ then passes only if

$$\textstyle\sum_{x \neq \alpha_w} d[x]\,(\delta_x - \delta_{\alpha_w})\,A_f[x] = 0.$$

For any $d$ that touches a row with $A_f[x] \neq 0$ this is a non-zero linear form in independent uniform keys, which vanishes with probability $1/p$. What remains undetected is a deviation concentrated on the queried row itself, which rescales the answer; because the provider's view is independent of $\alpha_w$, it succeeds with probability at most $\max_x \Pr[\alpha_w = x]$, the predictability of the query. The reference implementation uses per-row keys on the fast path by default, and Section 9.14 measures them against the single key. They do not carry over to the flexible path, whose contraction runs over records rather than tags; there the single $\delta$ remains.

## 5.9. Record retrieval and query authorisation

When the DU needs payloads, providers return $b_j$ directly; the DU subtracts $P[\alpha_w]$, unpacks $b$, and fetches the payloads with a batch of $k_{\max}$ DPF retrievals padded to a public bound, as in Splinter [12]. The response is $\lceil N_d/32\rceil$ words whatever $|S|$.

Authorisation is easy to overstate. DPF privacy hides the queried row, so no provider can check *which* keyword, cell or range a query targets, and a client permitted any query can read any row; keys that are not point functions (an $n$-party key *is* a share vector, and a two-party key is pseudorandom) give arbitrary linear combinations of rows, columns or cube cells, and the flexible path's second round is by construction an oracle for any linear function of the record column. For a fully trusted DU none of this exceeds its authority. For a DU entitled to some keywords, templates or cubes and not others, the remedy is a proof carried with the key, as in PACL [21], or a verifiable DPF [22]; the constructions we know [20] have the servers compare a short message, relaxing no-interaction to one short exchange per query. We state per-user authorisation as a requirement, not a solved problem.

### 5.10. The flexible path: conjunctions and ad-hoc weights

Registered single-keyword templates cover the common case. For anything else, such as a conjunction or disjunction of keywords, a weight vector chosen at query time, or a threshold on the matched set, the DU runs two rounds.

*Round one* is a retrieval (Section 5.9) for each keyword involved, after which the DU holds the posting lists and forms whatever selection vector it needs over the records, for example $v = w \circ (b_{w_1} \wedge b_{w_2})$ for an ad-hoc weight vector $w$ and a conjunction of two keywords.

*Round two* deals $v$ as fresh additive shares $v = \sum_j v_j$ over $\mathbb{Z}_p$, compressed exactly like the $n$-party key of Section 3: $n-1$ providers receive a $\lambda$-bit seed whose PRF expansion is their share, and one receives the explicit difference vector. Each provider runs the contraction of Section 5.3 against the record column, packed with $m_i$ in row 0 and $\delta m_i$ in row 1 of $\lceil N_d/(\ell/2) \rceil$ ciphertexts, and returns one ciphertext. The DU adds the $n$ responses, decrypts once and checks $\hat{y} = \delta y$. The magnitude bound of Lemma 1 is the DU's to check, against a public bound on $m_i$, since it alone knows $v$.

Round two costs $\lceil N_d/(\ell/2) \rceil$ plaintext-ciphertext products and $\log_2(\ell/2)$ rotations per provider, $\lambda$ bits of uplink to $n-1$ providers and $N_d \lceil \log p \rceil$ bits to one, and one ciphertext down. Every share a coalition of $n-1$ providers holds is uniform, or pseudorandom for the seeds, independently of $v$, so round two adds to the coalition's view nothing that depends on the query (Theorem 3). The DU learns $S$ and $|S|$, which $\mathcal{L}^{\mathrm{DU}}$ already contains.

A single round is not possible here in general: a conjunction is the product of two selection vectors, which no linear contraction of shares computes, and an ad-hoc weight vector would need a mask correction the DO cannot precompute (Remark 1).

### 5.11. Dynamic ingest

IoT streams are append-only. CR-SHE keeps a static tier and a hot buffer of at most $B$ records scanned by the same contraction, folded into a rebuilt tier every $B$ insertions. Because the coalition's per-query view is independent of the query (Theorem 3), forward and backward privacy hold trivially. Three rules keep updates from leaking. *Rows never move*: a new keyword takes the next free, masked, all-zero row of $D$, so only exhausting the provisioned headroom is visible. *A fresh mask key per static tier*: a rebuild re-lays the record axis, and reusing $K'$ would expose the change in membership. *Whole columns are replaced*: re-encrypting only the affected ciphertexts of $A_f$ would reveal which blocks of tags changed; summed-area tables with time as the first axis are the exception (Section 5.7), since only the newest, publicly known slice changes.

### 5.12. Worked instance

A clinician queries the tag `arrhythmia` with the registered template MEAN. The DU derives $\alpha_w$ and sends one DPF key per provider. Each provider performs a full-domain pass, contracts against the SUM and COUNT columns packed side by side, and returns one ciphertext carrying both values and both MACs. The DU adds the shares, decrypts once, verifies the MACs, and divides. No provider, and no coalition of up to $n-1$, learns the tag queried, which records contributed, how many did, or the mean itself.

**Algorithm 1** Outsourcing at the data owner.

**procedure** Setup$(1^\lambda, n, t, p)$
  $D \leftarrow \emptyset$;  $K', K'' \leftarrow \{0,1\}^\lambda$;  $\delta \leftarrow \mathbb{Z}_p^*$
  $(pk, sk) \leftarrow \mathsf{KGen}(1^\lambda, L_{\mathrm{HE}}, p, \ell)$
  $pp \leftarrow (n, t, N, N_d, p, \ell, \{w_f\})$
  **return** $(D, K', \delta, pk, sk, pp)$
**end procedure**

**procedure** BuildIndex$(D, K', DB)$
  $D(w) \leftarrow |D|$ for each new $w$ ▷ rows never move
  $I[D(w)][i] \leftarrow 1$ for each keyword $w$ of record $i$
  $P[x] \leftarrow$ rejection-sampled $F'_{K'}(x, \cdot)$
    uniform on $\mathbb{Z}_p^{N_d}$, for all $x \in [N]$
  $I' \leftarrow I + P \bmod p$
  **return** $I'$ ▷ replicated to all $\mathrm{CP}_j$
**end procedure**

**procedure** Precompute$(pk, K', DB, \{w_f\})$
  **for all** registered templates $f$ **do**
    $A_f[x] \leftarrow \sum_i I[x][i]\, w_f[i]\, m_i$
    $\widehat{A}_f[x] \leftarrow \delta_x A_f[x]$,  $\delta_x \leftarrow F''_{K''}(x)$
    pack $A_f$ in row 0, $\widehat{A}_f$ in row 1,
      templates in segments (§5.6); encrypt
  **end for**
**end procedure**

**procedure** EncData$(pk, DB)$
  pack $m_i$ in row 0, $\delta m_i$ in row 1; encrypt
    $\lceil N_d/(\ell/2) \rceil$ ciphertexts
  symmetric-encrypt opaque payloads
**end procedure**

## 6. Security analysis

*Leakage profile.* We define $\mathcal{L} = (\mathcal{L}_{\mathrm{Setup}}, \mathcal{L}^{\mathrm{CP}}, \mathcal{L}^{\mathrm{DU}})$ with an observer split:

$$\begin{aligned}\mathcal{L}_{\mathrm{Setup}}(DB) &= (n, t, N, N_d, p, \ell, \{w_f\}, \{(s_1, \ldots, s_d)\}_{\mathrm{cubes}}),\\ \mathcal{L}^{\mathrm{CP}}_{\mathrm{Query}} &= \mathrm{class}(q),\\ \mathcal{L}^{\mathrm{DU}}_{\mathrm{Query}} &= \big(|S|,\ f(\mathrm{records}(S))\big).\end{aligned}$$

Here class$(q)$ says whether a query is a fast-path aggregate over a named layout of registered templates, a range aggregate over a named cube and layout, a retrieval, or the second round of a flexible query. The coalition learns that and the public dimensions of the deployment, and nothing else: not the keyword, not the range or whether it touches the cube's edge, not the access pattern, not $|S|$, not the weights, not the result. In contrast to a design in which the matched set is returned to the providers for evaluation, and in contrast to an unmasked index, $\mathcal{L}_{\mathrm{Setup}}$ here contains no posting-list lengths and no membership structure, by Theorem 1. Retrieval adds the public padding bound $k_{\max}$ to $\mathcal{L}^{\mathrm{CP}}$. A conjunctive query is two retrievals and one flexible round, adding only result sizes to $\mathcal{L}^{\mathrm{DU}}$, never to the coalition.

**Definition 1 ($\mathcal{L}$-security against $t$-collusion).** In $\mathbf{Real}_{\mathcal{A},\Pi}(1^\lambda)$: (1) $\mathcal{A}$ outputs $(DB, C)$ with $|C| \le t$; (2) the challenger runs Setup, BuildIndex, Precompute, EncData and gives $\mathcal{A}$ the static view $(I', \mathrm{CT}, \widehat{\mathrm{CT}}, \{A_f, \widehat{A}_f\}_f, \{T_f, \widehat{T}_f\}_{f,\mathrm{cube}}, pk, pp)$; (3) for $q = 1, \ldots, Q$, $\mathcal{A}$ adaptively issues a query (a keyword with a template layout, a box of a registered cube with a layout, a keyword to retrieve, or a selection vector $v_q$ for a flexible round) and receives the coalition's per-query view: its DPF keys $\{\kappa_{q,j}\}_{j\in C}$ or its shares $\{v_{q,j}\}_{j\in C}$, and the responses those providers compute; (4) $\mathcal{A}$ outputs a bit. In $\mathbf{Ideal}_{\mathcal{A},\mathcal{S},\mathcal{L}}(1^\lambda)$ a PPT simulator $\mathcal{S}$ produces the views given only $\mathcal{L}_{\mathrm{Setup}}$ and the per-query leakage. $\Pi$ is $\mathcal{L}$-adaptively-secure against $t$-collusion if for every PPT $\mathcal{A}$ corrupting $\le t$ providers there is a PPT $\mathcal{S}$ with $|\Pr[\mathbf{Real} = 1] - \Pr[\mathbf{Ideal} = 1]| \le \mathsf{negl}(\lambda)$.

**Algorithm 2** Query, at the data user and at provider $\mathrm{CP}_j$.

**procedure** $\mathsf{Token}(D, w, f)$ ▷ at the DU
  $\alpha_w \leftarrow D(w)$
  $(\kappa_1, \ldots, \kappa_n) \leftarrow \mathsf{Gen}(1^\lambda, \alpha_w, 1)$ over $\mathbb{Z}_p$
  **return** $(\kappa_j, f) \rightarrow \mathrm{CP}_j$
**end procedure**

**procedure** $\mathsf{FastAgg}(\kappa_j, f, A_f, \widehat{A}_f)$ ▷ single keyword
  check the shape of $\kappa_j$ ▷ §5.9
  $e_j[x] \leftarrow \mathsf{Eval}(j, \kappa_j, x)$, $x \in [N]$ ▷ $O(N)$ PRG
  $\mathrm{ct}_j \leftarrow \boxplus_c E_{j,c} \boxdot \mathrm{A}_c$ ▷ $O(N/\ell)$ HE
    $E_{j,c}$: chunk $c$ of $e_j$ in both rows
  rotate and sum within segments; compress
  **return** $\mathrm{ct}_j$ ▷ one ct, value + MAC
**end procedure**

**procedure** $\mathsf{Deal}(v)$ ▷ at the DU, flexible
  $v = \sum_j v_j$: seeds to $n-1$ providers,
    explicit difference to one
**end procedure**

**procedure** $\mathsf{FlexAgg}(v_j, \mathrm{CT})$ ▷ round two
  $\mathrm{ct}_j \leftarrow \boxplus_c V_{j,c} \boxdot \mathrm{CT}_c$; rotate and sum
  **return** $\mathrm{ct}_j$
**end procedure**

**procedure** $\mathsf{Retrieve}(\kappa_j, I')$
  **return** $b_j \leftarrow \sum_x e_j[x] \cdot I'[x]$ ▷ fixed size
**end procedure**

**procedure** $\mathsf{CombineDec}(sk, K', \delta, \cdot)$ ▷ at the DU
  $(y, \hat{y}) \leftarrow$ slots $(0, \ell/2)$ of $\mathsf{Dec}(sk, \boxplus_j \mathrm{ct}_j)$
  **abort** if $\hat{y} \neq \delta_{\alpha_w} y$ (fast) or $\delta y$ (flexible)
  retrieval: $b \leftarrow \mathrm{unpack}\big((\sum_j b_j) - P[\alpha_w]\big)$
    $S \leftarrow \{i : b[i] = 1\}$
  **return** $y$, and $S$ if retrieving
**end procedure**

**Theorem 2 (Correctness).** *Fix a database $DB$, a keyword $w$, and a registered template $f$ satisfying the magnitude bound of Lemma 1. Then* CombineDec *returns $y = \sum_{i \in S} w_f[i]\, m_i$ with $S = \{i : I[\alpha_w][i] = 1\}$ on the fast path, a range query on a box $B$ of a registered cube whose total is below $p$ returns $\sum_{i:\, c(i) \in B} w_f[i]\, m_i$,* Retrieve *returns exactly $S$, and a flexible round on a selection vector $v$ satisfying the magnitude bound returns $\sum_i v[i]\, m_i$, except with the negligible probability that leveled-HE decryption fails.*

Proof. By DPF correctness $\sum_j e_j[x] = 1$ at $x = \alpha_w$ and 0 elsewhere.

*Fast path.* $\boxplus_j \mathrm{ct}_j$ decrypts, by Lemma 1 applied with $\beta = \big(\sum_j e_j\big)$ over the array $A_f$, to $A_f[\alpha_w] = \sum_i I[\alpha_w][i] w_f[i] m_i = \sum_{i \in S} w_f[i] m_i$.

*Range.* The $2^d$ keys of a range query share the point functions $\beta_\sigma\, u_{\kappa_\sigma}$, with $\beta_\sigma = 0$ for a corner outside the cube. In the combined layout $\sum_j e_j = \sum_\sigma \beta_\sigma u_{\kappa_\sigma}$, so by Lemma 1 applied over the column $T_f$ the response decrypts to $\sum_\sigma \beta_\sigma T_f[\kappa_\sigma]$; in the per-corner layout segment $\sigma$ decrypts to $\beta_\sigma T_f[\kappa_\sigma]$ and the DU adds them. By (1) that sum is the box aggregate modulo $p$. The true aggregate lies in $[0, p)$ because it is at most the cube's total, so the reduction loses nothing, even though individual signed terms are negative.

*Retrieval.* $\sum_j b_j = \sum_x \big(\sum_j e_j[x]\big) I'[x] = I'[\alpha_w] = \mathrm{pack}(b) + P[\alpha_w]$, so $\big(\sum_j b_j\big) - P[\alpha_w] = \mathrm{pack}(b)$ exactly, and the DU, which derives $P[\alpha_w]$ from $K'$, unpacks $b$.

*Flexible round.* $\sum_j v_j = v$, so by Lemma 1 applied with $\beta = v$ over the record column, $\boxplus_j \mathrm{ct}_j$ decrypts to $\sum_i v[i]\, m_i$ in slot 0 and $\delta \sum_i v[i]\, m_i$ in slot $\ell/2$.

*Packing.* Rotations act within slot rows and the rotate-and-sum covers one segment, so the slot that carries template $r$ receives only the products of template $r$'s segment, and the value and MAC rows do not mix.

The dictionary is injective by construction, so row $\alpha_w$ is exactly the records containing $w$. Decryption correctness under the noise growth discussed in the proof of Lemma 1 contributes the negligible failure term. The MAC check passes with certainty for honest providers, since $\delta$, or $\delta_{\alpha_w}$ on the fast path, distributes over the linear contraction: $\sum_x e[x]\,\delta_x A_f[x] = \delta_{\alpha_w} A_f[\alpha_w]$ when $e = u_{\alpha_w}$. □

**Theorem 3 ($t$-collusion leakage security).** *If $F'$ is a secure PRF, $(\mathsf{Gen}, \mathsf{Eval})$ is an $(n-1)$-private DPF, and the homomorphic and symmetric schemes are IND-CPA, then CR-SHE is $\mathcal{L}$-adaptively-secure against $t$-collusion in the sense of Definition 1, for $t = n-1$ and $\mathcal{L}^{CP}_{\mathrm{Query}} = \mathrm{class}(q)$. Concretely, for every* PPT $\mathcal{A}$ *there exist* PPT $\mathcal{S}$ *and $B_1, \ldots, B_4$ with*

$$\big|\Pr[\mathbf{Real}{=}1] - \Pr[\mathbf{Ideal}{=}1]\big| \le \mathrm{Adv}^{\mathrm{PRF}}_{B_1} + Q \cdot \big(2^{d_{\max}} \mathrm{Adv}^{\mathrm{DPF}}_{B_2} + \mathrm{Adv}^{\mathrm{PRF}}_{B_4}\big) + m \cdot \mathrm{Adv}^{\mathrm{CPA}}_{B_3},$$

*where $m$ is the number of ciphertexts in the coalition view. As is standard for BFV, IND-CPA is assumed to hold even given the relinearisation and rotation keys the providers need (circular security). The theorem is stated for one static tier; with a fresh $K'$ per tier (Section 5.11) it applies to each tier independently, with one PRF term per tier.*

Proof. Take $|C| = n-1$ without loss of generality; a smaller coalition holds a subset of the same state. Let $H_0 = \mathbf{Real}$.

*$H_1$ (PRF to random, index to zero).* Replace $F'$ by a truly random function, at cost $\mathrm{Adv}^{\mathrm{PRF}}_{B_1}$. The mask is then an exactly uniform one-time pad (Theorem 1), so building $I'$ from the all-zero index instead of $I$ changes nothing; $I'$ now depends only on $(N, N_d)$.

*$H_2$ (DPF keys to simulated).* Replace every DPF key the coalition holds by $\mathcal{S}_{\mathrm{DPF}}(1^\lambda, C, \cdot)$ for its domain, licensed by $(n-1)$-privacy, which hides the output $\beta$ as well as the point. A range query to a $d$-dimensional cube carries $2^d$ independent keys, dummy keys included, so a hybrid over all keys costs $Q\,2^{d_{\max}}\mathrm{Adv}^{\mathrm{DPF}}_{B_2}$, where $d_{\max}$ is the largest registered cube dimension. The per-query key material now depends only on $C$, the domain and the query class; in particular it no longer depends on which corners were dummies.

*$H_2'$ (flexible shares to uniform).* In each flexible round at least one share lies outside $C$. If it is the explicit difference vector, the coalition's shares are seed expansions independent of $v_q$. If it is a seed's expansion, the explicit vector is $v_q$ minus a PRF output the coalition cannot compute, and replacing that output by a uniform vector costs $\mathrm{Adv}^{\mathrm{PRF}}_{B_4}$. Either way the coalition's shares become uniform and independent of $v_q$, over at most $Q$ rounds.

*$H_3$ (ciphertexts to encryptions of 0).* Replace each of the $m$ ciphertexts in the coalition view, namely the record column, the columns $A_f, \widehat{A}_f$ and the tables $T_f, \widehat{T}_f$, by encryptions of 0 under $pk$, at cost $m \cdot \mathrm{Adv}^{\mathrm{CPA}}_{B_3}$, since the coalition holds only $pk$.

*Simulator.* The responses $\mathrm{ct}_{q,j}$ need not be simulated separately: each is a deterministic function of $\kappa_{q,j}$ or $v_{q,j}$, the query class, and the provider's stored state, all of which $\mathcal{S}$ possesses in $H_3$. Compression is a public, deterministic map and changes nothing. $\mathcal{S}$ therefore regenerates the entire coalition view from $\mathcal{L}_{\mathrm{Setup}}$ and $\mathrm{class}(q)$ alone, so $\mathbf{Ideal}_{\mathcal{A},\mathcal{S},\mathcal{L}}$ matches $H_3$; summing the terms proves the theorem. □

**Lemma 2 (Per-corner verification).** *In the per-corner layout, let provider $j$ contract segment $\sigma$ against $e_{\sigma,j} + d_\sigma$ in both rows, for vectors $d_\sigma$ of its choice, and let the per-cell keys $\delta_c$ be uniform and unknown to it. The DU accepts a result different from the honest one only if, for some $\sigma$, $d_\sigma$ is supported on $\{\kappa_\sigma\} \cup \{c : T_f[c] = 0\}$ for a real corner, or on $\{c : T_f[c] = 0\}$ for a dummy corner; otherwise it accepts with probability at most $1/p$.*

Proof. Segment $\sigma$ decrypts to $y = \beta_\sigma T_f[\kappa_\sigma] + \Delta$ and $\hat{y} = \beta_\sigma \delta_{\kappa_\sigma} T_f[\kappa_\sigma] + \hat{\Delta}$, with $\Delta = \sum_c d_\sigma[c] T_f[c]$ and $\hat{\Delta} = \sum_c d_\sigma[c]\delta_c T_f[c]$. For a real corner the check $\hat{y} = \delta_{\kappa_\sigma} y$ passes iff $\sum_{c \neq \kappa_\sigma} d_\sigma[c](\delta_c - \delta_{\kappa_\sigma}) T_f[c] = 0$. If $d_\sigma$ touches some $c \neq \kappa_\sigma$ with $T_f[c] \neq 0$, this is a non-zero affine form in the independent uniform $\delta_c$ and vanishes with probability $1/p$. For a dummy corner the DU requires $y = \hat{y} = 0$, so $\hat{\Delta} = 0$, which has probability $1/p$ unless $d_\sigma[c] T_f[c] = 0$ for all $c$. The DU accepts only if every segment passes, so one segment that violates the support condition already bounds acceptance by $1/p$. If every $d_\sigma$ satisfies the support condition, each perturbation is either zero or concentrated on $\kappa_\sigma$, the corner the DPF hides. □

Because cell aggregates are non-negative, $T_f[c] = 0$ only when the box $\prod_k [0, c_k]$ holds no record with a non-zero contribution (for COUNT, no record at all), so in practice an undetected deviation must hit the hidden corner itself. Its success probability is then at most the provider's chance of guessing $\kappa_\sigma$.

**Remark 3 (Rounds and leakage).** Suppose the client recovers $S$ and then requests $\mathsf{Compute}(pk, \{\mathrm{CT}_i\}_{i \in S}, f)$ from the providers. The coalition's per-query view then contains $S$ itself, which is a function of the queried keyword and the database. No simulator given only data-independent leakage can produce it, so an empty per-query leakage profile is *unattainable* in that design rather than merely loose. The flexible path has the same two rounds, but its second round sends fresh shares of the selection instead of the selection, and $H_2'$ shows those are simulatable. What makes Theorem 3 true is contracting a share at the provider, not the number of rounds.

**Remark 4 (Confidentiality and degradation).** For two databases of equal public dimensions differing only in contents, Theorem 3 makes each coalition view indistinguishable from a distribution depending only on $\mathcal{L}_{\mathrm{Setup}}$, which is identical for both; the executions differ by at most $m \cdot \mathrm{Adv}^{\mathrm{CPA}}$, with the content dependence isolated in $H_3$. The same proof applies for any $|C| \le n - 1$, since $(n-1)$-privacy holds for every strict subset; a coalition of all $n$ providers is outside the model, and against it the guarantee degrades to IND-CPA confidentiality of the records with the index still masked under $F'$.

**Remark 5 (Beyond semi-honest).** Section 5.8 detects a response inconsistent with its MAC except with probability $1/p$. It does not detect a provider that applies the same linear substitution to both halves, that is, a contraction against any vector other than its evaluation share, nor does it guarantee output delivery. Full malicious security through verifiable function secret sharing remains future work, which is why Table 1 records *detect* rather than ✓.

## 7. Complexity analysis

Table 3 summarises the asymptotic costs. Four entries deserve comment because they correct the intuitive accounting.

First, the selection contraction of a retrieval, and hence of the flexible path's first round, is $\Theta(NN_d)$ ring operations, not $\Theta(N)$: a full-domain evaluation produces $N$ share values, each of which multiplies an index *row* of width $N_d$. Counting only the $O(N)$ pseudorandom-generator calls of the evaluation omits the dominant term. With a small modulus and vectorised arithmetic these are cheap word operations, and the pass is embarrassingly parallel, but the asymptotic is what it is: this is the ORAM and PIR lower-bound price for hiding the access pattern from a coalition without preprocessing [24, 40]; preprocessing PIR avoids it at the cost of client hints that updates invalidate [32].

Second, the fast path of Section 5.5 removes the $N_d$ factor entirely, at $O(N)$ generator calls plus $O(N/\ell)$ packed ciphertext operations, at the cost of restricting queries to single keywords and templates registered at setup.

Third, the aggregate response is one ciphertext, for up to $\ell/2$ templates, and, crucially, *independent of* $|S|$. A response whose size grew with the result set would leak the result-set size to the coalition and contradict $\mathcal{L}^{\mathrm{CP}}$. Retrieval responses are $\lceil N_d/32 \rceil$ words of fixed size for the same reason. Range queries likewise return one ciphertext, for any box.

Fourth, the single-server alternative of Section 5.6 needs no non-collusion assumption but pays in two places. On the fast path its query is $\lceil N/(\ell/2) \rceil$ fresh ciphertexts instead of an $O(\lambda \log N)$ key, and its contraction uses ciphertext-ciphertext products. On retrieval and the flexible path it must multiply an encrypted $N$-vector by the packed $N \times \lceil N_d/32 \rceil$ index, about $N\lceil N_d/32 \rceil/\ell$ plaintext-ciphertext products with diagonal packing [64], where CR-SHE does $\Theta(NN_d/32)$ word operations. Both are linear in $NN_d$: the gap is the constant ratio of a homomorphic slot operation to a machine-word operation, which Section 9.6 bounds from below. What does not shrink with better kernels is the client side: a $\lambda \log N$-bit key against $\lceil N/(\ell/2) \rceil$ fresh encryptions.

**Table 3**
Asymptotic costs. $\gamma$ is the per-provider DPF key size, $O(\lambda \log N)$ at $n = 2$; $\ell$ is the SIMD slot count.

| Resource | Per provider | Data user |
|---|---|---|
| Index storage | $O(NN_d \log p/32)$, packed | n/a |
| SAT storage, $d$-cube | $O(M/\ell)$ (comb.), $O(2^d M/\ell)$ | n/a |
| Ciphertext storage | $O(N_d/\ell) + O(N/\ell)$ per $f$ | n/a |
| Query uplink | $O(\gamma)$ | $O(n\gamma)$ |
| Flexible uplink | $\lambda$, or $N_d\lceil \log p \rceil$ bits | $O(N_d \log p)$ |
| Aggregate downlink | 1 ct | $n$ ct |
| Retrieval downlink | $\lceil N_d/32 \rceil$ words, fixed | $O(nN_d)$ bits |
| Fast path | $O(N)$ PRG, $O(N/\ell)$ HE | 1 decrypt |
| Range, $d$-cube, $M$ cells | $2^d \cdot O(M)$ PRG, $O(kM/\ell)$ HE | $2^d$ keygen, 1 decrypt |
| Range uplink | $2^d\gamma$ | $n2^d\gamma$ |
| Retrieval / round one | $O(N)$ PRG, $\Theta(NN_d/32)$ ring | $O(N_d)$ |
| Flexible round two | $O(N_d/\ell)$ HE, $O(\log \ell)$ rot. | 1 decrypt |
| Owner precomputation | postings per $f$; $O(NN_d)$ mask | n/a |
| Insertion, $\tau$ tags | $O(\tau)$ updates, $O(N/\ell)$ re-enc. | n/a |

## 8. Implementation

Two implementations share the parameters below. A C++17 prototype implements the single-keyword path in the two-ciphertext layout with a single MAC key; it produced the measurements on the real corpora (fast path, deployment, leakage), reported where the measured quantity does not depend on the later layout. An open reference implementation (Python driving OpenFHE's C++ kernels) implements the full construction of Section 5: range aggregation over data cubes in both layouts, the packed response, per-row and per-cell MAC keys, the packed and masked index with rejection-sampled masks, retrieval, the flexible path, and baseline (vi). Its 57 tests include DPF checks at every point of every domain up to depth 8, range answers against brute force for $d \leq 3$ and $n \leq 3$, two failure controls that must and do produce wrong answers (a flipped corner sign; an out-of-cube corner treated as real), and tampering tests for every MAC; six deliberate source mutations of the range code are all caught.

*Primitives.* The two-party DPF is the Boyle–Gilboa–Ishai GGM-tree construction [17] with fixed-key AES-128 in the Matyas–Meyer–Oseas form $G(s) = \mathrm{AES}_k(s) \oplus s$ under three fixed keys, evaluated over the full domain in one pass. The homomorphic layer is leveled BFV in OpenFHE [65] (1.2.3 in C++, 1.5.1 in Python) at plaintext modulus $p = 68{,}724{,}326{,}401$, a 37-bit prime with $p \equiv 1 \bmod 2^{17}$, ring dimension 8,192, multiplicative depth 1, at the 128-bit classical security level. In both implementations the DPF output group and the BFV plaintext ring are one modulus object, so the composition error that Lemma 1 warns about cannot be written, and a template or cube whose total would reach $p$ is rejected at setup.

*Corpora.* The primary corpus is the Environmental Sensor Telemetry dataset of 405,184 MQTT messages from three sensor arrays [66], indexed under the device, a one-minute window, decile bands of five sensor channels and two boolean flags: $N = 11{,}580$ tags over 3,646,656 postings (median 35, mean 314.9, maximum 404,702), aggregating temperature in hundredths of a degree. The second is the UCI Beijing Multi-Site Air-Quality dataset [67, 68]: 420,370 hourly records from twelve sites after dropping 398 with a missing temperature, $N = 35{,}162$ tags. The range experiments use a synthetic cube with the primary corpus's dimensions (Section 9.2).

*Baselines.* (i) a plaintext scan; (ii) single-server searchable encryption, which leaks search and access patterns and $|S|$; (iii) Path ORAM with $Z = 4$; (iv) a DORY-style [13] search-only baseline, our selection stopped at the selection vector; (v) a faithful homomorphic equality scan, $\mathrm{eq}(a, b) = 1 - (a - b)^{q-1}$, measured on one batch and extrapolated (marked †); and (vi) a single server holding the same encrypted columns, queried with an encrypted one-hot vector, which needs no non-collusion assumption and so isolates what that assumption buys.

*Machines.* C++ measurements: one AMD EPYC 7J13, 64 cores, 64 GB, medians of 11 runs unless stated. Reference implementation: a cloud VM with four vCPUs of an Intel Xeon at 2.1 GHz with AES-NI, 16 GB, one OpenMP thread, medians of 5–21 runs; every JSON output carries this environment stamp, and every table built from them is generated by a script rather than transcribed. The two sets of latencies are not

comparable with each other. The C++ fast-path figures use the two-ciphertext layout and so bound the packed layout from above.

## 9. Results analysis and discussion

### 9.1. Correctness

Every result below comes from an implementation that first has to compute the right answer. The C++ prototype matched the plaintext ground truth exactly, with the MAC verifying, for 1,000 random tags per template and $n \in \{2, 3, 4\}$ on both real corpora, returned exactly the posting list on retrieval, and sent a byte-identical response for matched sets of size 0 and 4,999. The reference implementation matches the ground truth for every range query in its benchmarks and tests, and, on a synthetic corpus shaped like the primary one, for 200 random tags at each $n \in \{2, 3, 4\}$ on the fast path, retrieval and the flexible path with ad-hoc weights, with a byte-identical 132,081 B response for the shortest and longest posting lists.

### 9.2. Range aggregation

*Setup.* The range path is measured in the reference implementation on a telemetry-shaped cube of 3 devices, 11,523 one-minute windows and 10 value bands of one sensor channel, $M = 345{,}690$ cells over 405,184 records, with templates COUNT and SUM. The records are synthetic, generated with the primary corpus's dimensions, because the raw corpora could not be downloaded into the environment that produced these measurements; a range query's cost depends only on the cube's dimensions, and every answer below was checked against the plaintext ground truth on these records. A query fixes one device, a window of $L$ minutes and bands 3–7, so $d = 3$ and each provider receives 8 keys. One thread, medians of 5 runs, interquartile ranges in the result files.

**Table 4**
Range aggregation per provider against window length $L$, $M = 345{,}690$ cells, $d = 3$, two templates. Reference implementation, one thread. Every query sends 2,760 B of keys and returns 132,081 B.

| $L$ (min) | Per-corner layout (ms) | | Combined layout (ms) | |
|---|---|---|---|---|
| | DPF eval | Provider total | DPF eval | Provider total |
| 1 | 547.3 | 3290.9 | 489.6 | 891.7 |
| 60 | 522.2 | 3436.1 | 446.7 | 913.7 |
| 240 | 501.0 | 3240.6 | 447.8 | 781.5 |
| 1,440 | 597.4 | 3253.3 | 512.1 | 810.2 |
| 11,523 | 515.6 | 3298.1 | 452.3 | 843.7 |

*Cost does not depend on the range* (Table 4). From a one-minute window to the whole 11,523-minute timeline, a provider spends 3.24–3.44 s in the per-corner layout and 0.78–0.91 s in the combined layout, of which 0.45–0.60 s is the evaluation of the eight DPF keys. Every query sends 2,760 B of keys to each provider and receives one 132,081 B ciphertext. The whole-timeline box already starts at the cube's edge, so four of its eight keys are dummies; a 240-minute box at the cube's origin, on which seven of the eight are dummies, costs the same (3,238.8 and 937.8 ms) and sends the same eight keys. The data user spends 6–15 ms generating keys and 1.1–1.9 ms combining, verifying and decrypting.

*Where the time goes.* The per-corner layout stores each table eight times and contracts 1,351 packed chunks, the combined layout 169. At the measured 0.092 ms per plaintext-ciphertext product (Section 9.6) the products themselves account for 125 and 16 ms, a computed figure; the rest is the DPF evaluation and the Python-side encoding of one plaintext vector per chunk, which a native implementation largely avoids. The per-corner layout buys verification of every corner (Lemma 2); the combined layout buys a fourfold lower provider cost and an eightfold smaller encrypted table.

*Cost grows with the cube, linearly* (Table 5): 34.7, 129.3 and 1,356.0 ms per provider for $10^4$, about $10^5$ and $10^6$ cells, so cube size, not record count or range length, is the quantity a deployment provisions. The owner chooses which families form a cube, as it chooses templates.

*Against the alternatives.* The same query through single-tag aggregates, with each cell registered as its own tag, needs one query per cell in the box: 1,200 queries for a 240-minute window over five bands, each

**Table 5**
Range aggregation against cube size, $d = 2$, combined layout, one template. Reference implementation, one thread.

| Cells $M$ | Chunks | DPF eval (ms) | Provider total (ms) |
|---|---|---|---|
| 10,000 | 3 | 8.3 | 34.7 |
| 99,856 | 25 | 60.9 | 129.3 |
| 1,000,000 | 245 | 722.2 | 1356.0 |

with its own keys and response, and the number of queries reveals the box's size. Through the flexible path it needs a retrieval for the device, each of the 240 minutes and each band before the contraction. A single server holding the same encrypted table needs no non-collusion assumption but an encrypted query as long as the table: 169 fresh ciphertexts in the combined layout, 44.4 MB of uplink computed from the measured ciphertext size, against 2.7 kB of DPF keys.

## 9.3. Selection latency and key size

Table 6 reports the per-provider full-domain evaluation in the C++ prototype, which is the whole of the symmetric-key selection. At $N = 11{,}580$ it costs 0.336 ms on one thread; at $N = 10^6$, 45.7 ms on one thread and 18.8 ms at thirty-two, beyond which it regresses as thread dispatch outweighs the per-level work. The two-party key grows from 266 B to 368 B, as $O(\lambda \log N)$ predicts. Sub-millisecond cells vary by up to 25% between runs, so only orders of magnitude are read from them: selection is not the bottleneck; the homomorphic contraction is.

**Table 6**
Per-provider selection latency and two-party key size against $N$. Medians of 21 runs.

| $N$ | Latency (ms), $T$ threads | | | | | Key |
|---|---|---|---|---|---|---|
| | $T = 1$ | 4 | 16 | 32 | 64 | (B) |
| 1,000 | 0.032 | 0.032 | 0.032 | 0.032 | 0.032 | 198 |
| 10,000 | 0.290 | 0.175 | 0.156 | 0.161 | 0.183 | 266 |
| 11,580 | 0.336 | 0.200 | 0.177 | 0.181 | 0.211 | 266 |
| 100,000 | 4.418 | 2.487 | 1.909 | 1.928 | 2.420 | 317 |
| $10^6$ | 45.741 | 25.701 | 19.711 | 18.769 | 24.401 | 368 |

## 9.4. Oblivious aggregation

In the C++ prototype the fast path costs 38.8–39.2 ms per provider across a 405× range of record counts, $N_d = 10^3$ to 405,184 (Table 7, Fig. 2); an earlier sweep in increasing $N_d$ showed a warm-up step near 53 ms that vanished when the order was reversed. The single-round general path of Remark 1, whose $\Theta(NN_d)$ pass is the flexible path's first round, cost 103.7 ms at $N_d = 10^3$ and 1,378.2 ms at the full corpus, 35.5× the fast path, with an unpacked masked index of 37.5 GB per provider, which packing divides by 32. The two-ciphertext response, 525,950 B, was byte-identical for an empty posting list and one matching 4,999 of 5,000 records; packed and compressed it is 132,081 B.

## 9.5. Comparison against baselines

Table 8 places the C++ fast path against the baselines on the primary corpus for a probe tag with the median $|S| = 35$. Plaintext scan and single-server searchable encryption are essentially free and leak everything or the access pattern and $|S|$; Path ORAM pays per matched record, so its cost and 373 kB downlink scale with $|S|$. The DORY-style search-only baseline costs 130.4 ms, 3.3× the fast path, because it materialises the selection vector the fast path never touches. The faithful homomorphic equality scan needs multiplicative depth 17 at ring dimension 32,768: 1.86 s per batch of 32,768 slots and 24.2 s per query, extrapolated, 617× the fast path. It is the right comparison for ad-hoc predicates on one server; for registered aggregates the stronger single-server alternative is baseline (vi), measured in Section 9.6.

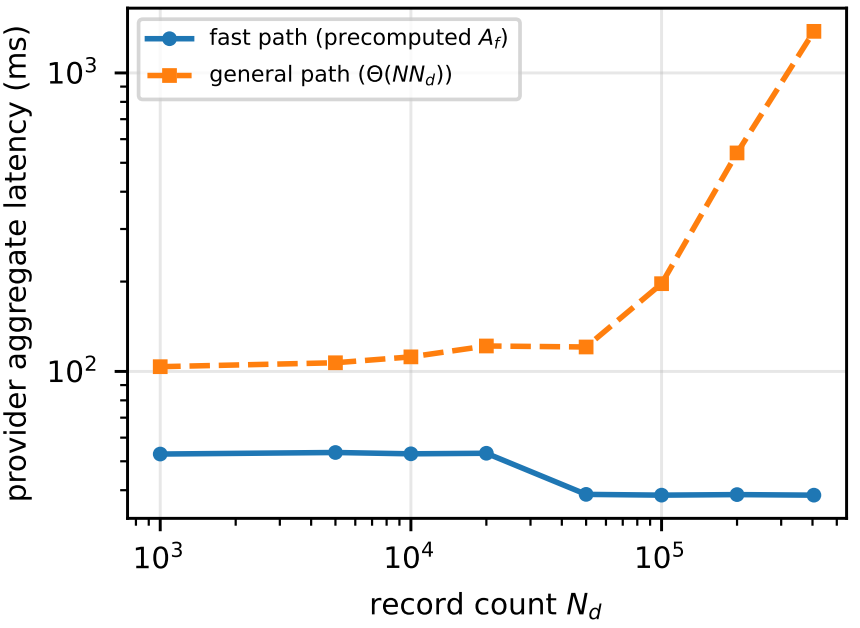



**Figure 2:** Aggregate latency, fast path against the single-round general path (C++, two-ciphertext layout). The $x$-axis is $N_d$, not $|S|$: cost no longer depends on the matched-set size. The fast path is flat to $\pm 0.5\%$ over a $405\times$ range.

**Table 7**
Aggregate latency and storage, fast path against the single-round general path, $N = 11{,}580$, template SUM, C++ prototype, two-ciphertext layout. Medians of 11 runs (fast) and 3 (general).

| $N_d$ | Fast (ms) | General (ms) | $I'$ / provider (GB) | Downlink (B) |
|---|---|---|---|---|
| $10^3$ | 38.8 | 103.7 | 0.09 | 525,950 |
| $5 \times 10^3$ | 39.0 | 106.9 | 0.46 | 525,950 |
| $10^4$ | 38.8 | 111.9 | 0.93 | 525,950 |
| $2 \times 10^4$ | 38.8 | 121.7 | 1.85 | 525,950 |
| $5 \times 10^4$ | 39.1 | 120.7 | 4.63 | 525,950 |
| $10^5$ | 39.2 | 196.8 | 9.26 | 525,950 |
| $2 \times 10^5$ | 39.1 | 539.5 | 18.53 | 525,950 |
| $4.05 \times 10^5$ | 38.8 | 1,378.2 | 37.54 | 525,950 |

## 9.6. Response packing, and what the trust split buys

Table 9 puts the two-ciphertext layout, the packed layout of Section 5.6, its compressed form and four-template responses through the reference implementation at both corpora's tag domains, together with baseline (vi), a single server holding the same encrypted columns. All rows come from one implementation on one machine and one thread, so they compare with each other and not with the C++ figures. Byte counts are a property of the parameters and of OpenFHE's serialisation, not of the machine; the C++ prototype's 262,975 B per ciphertext against 263,011 B here is a serialisation-version difference.

*Packing halves the provider's work and the downlink.* One rotate-and-sum over half-rows replaces two over full rows: a single template falls from 53.0 to 23.5 ms at $N = 11{,}580$ and from 55.8 to 31.4 ms at $N = 35{,}162$, and the response from two ciphertexts to one, 263,011 B, or 132,081 B compressed. Four templates in one compressed response cost 35.4 ms and 132,081 B at $N = 11{,}580$, against 194.9 ms and 2,104,088 B for four

**Table 8**
Baselines, same corpus and machine, $N_d = 10^5$, probe tag with the median $|S| = 35$. † marks an extrapolated value.

| System | Server (ms) | Downlink (B) | Leakage to the server(s) |
|---|---|---|---|
| Plaintext scan | $< 0.001$ | 8 | everything |
| SSE, 1 server | $< 0.001$ | 560 | search and access pattern, $\lvert S\rvert$ |
| Path ORAM | $0.059^{\dagger}$ | 373,248 | $\lvert S\rvert$, via access count |
| DORY-style search | 130.4 | 800,000 | none; no analytics |
| FHE equality scan | $24{,}157^{\dagger}$ | n/a | none; 1 server |
| **CR-SHE, fast** | **39.2** | **525,950** | $\mathrm{class}(q)$ only |

**Table 9**
Fast path per provider: two-ciphertext against packed layouts, and a single server on the same columns (baseline vi). Reference implementation, one thread; medians of 21 runs. $k$ is the number of templates answered.

| Layout | $k$ | Provider (ms) | Downlink (B) |
|---|---|---|---|
| $N = 11{,}580$ *tags* | | | |
| Two ct (unpacked) | 1 | 53.0 | 526,022 |
| Packed, 1 ct | 1 | 23.5 | 263,011 |
| Packed + compressed | 1 | 27.0 | 132,081 |
| 4× two ct | 4 | 194.9 | 2,104,088 |
| Packed, 1 ct | 4 | 35.4 | 263,011 |
| Packed + compressed | 4 | 32.7 | 132,081 |
| Single server (vi) | 1 | 42.6 | 263,211 |
| $N = 35{,}162$ *tags* | | | |
| Two ct (unpacked) | 1 | 55.8 | 526,022 |
| Packed, 1 ct | 1 | 31.4 | 263,011 |
| Packed + compressed | 1 | 36.4 | 132,081 |
| 4× two ct | 4 | 245.6 | 2,104,088 |
| Packed, 1 ct | 4 | 74.2 | 263,011 |
| Packed + compressed | 4 | 92.3 | 132,081 |
| Single server (vi) | 1 | 92.5 | 263,211 |

two-ciphertext responses (16× less downlink). At $N = 35{,}162$ they need 35 chunks instead of 12, so the cost grows to 74–92 ms while the response stays one ciphertext.

*Against a single server, the trust split buys the client side.* Baseline (vi) needs no non-collusion assumption. Its server costs 42.6 and 92.5 ms at the two tag domains, 1.8 and 2.9 times a packed CR-SHE provider, because its contraction multiplies ciphertext by ciphertext; its response is the same one ciphertext. Its query is not (Table 10): 789,033 and 2,367,099 B of fresh ciphertexts against a 260 or 294 B DPF key, 3,035 and 8,051 times more uplink, and 14.6 and 67.5 ms of client encryption against 0.54 and 0.62 ms of key generation. Query compression by ciphertext expansion [35] would cut that uplink to one ciphertext for more server automorphisms; we did not implement it.

**Table 10**
Per-query uplink and client cost: CR-SHE's two-party key against the single server's encrypted query. Reference implementation, one thread.

| | Key (B) | Enc. query (B) | Cts | Encrypt (ms) |
|---|---|---|---|---|
| $N = 11{,}580$ | 260 | 789,033 | 3 | 14.6 |
| $N = 35{,}162$ | 294 | 2,367,099 | 9 | 67.5 |

On retrieval a single server must bring every word of the packed $N \times \lceil N_d/32 \rceil$ index into contact with an encrypted one-hot query, at least $\lceil N\lceil N_d/32\rceil/\ell \rceil$ plaintext-ciphertext products before any rotation; packing helps it exactly as it helps CR-SHE. At the measured 0.092 ms per product that bound is 81.6 ms at $N_d = 2\times10^4$ and 1.65 s at $N_d = 405{,}184$, against 26.0 ms and 1.10 s for CR-SHE's packed retrieval (Table 12); across the sweep the ratio is 1.5–3.1×, and the two figures come from separate runs. The separation on every single-keyword path is therefore a small constant factor in server work, set by a homomorphic slot operation (11 ns here) against a word operation, plus three orders of magnitude in uplink. That is the honest case for the non-collusion assumption on these paths: it pays where the client is thin or the link is metered, which is the IoT setting. On ranges (Section 9.2) the gap is wider.

## 9.7. The flexible path

Table 11 reports the two rounds of a flexible query at $N = 11{,}580$ in the reference implementation, swept in decreasing $N_d$ to control for warm-up. Round one, the retrieval, grows linearly with $N_d$ as its $\Theta(NN_d)$ contraction predicts. Round two costs 20.1–35.6 ms with no trend in $N_d$ over this range, because one to five packed products are dwarfed by the twelve rotations of the row sum, and it returns one 132,081 B ciphertext whatever the selection. Its uplink is a 16 B seed to $n-1$ providers and $8N_d$ B to one. Every flexible query in the reference implementation's tests, including conjunctions, disjunctions and ad-hoc weights, decrypts to the plaintext ground truth.

**Table 11**
Flexible path, $N = 11{,}580$, $n = 2$, reference implementation, one thread. Round-two uplink is the heavy provider's; the others receive 16 B.

| $N_d$ | Round 1 (ms) | Round 2 (ms) | Up (B) | Down (B) | $I'$ (GB) |
|---|---|---|---|---|---|
| 1,000 | 41.2 | 20.1 | 8,000 | 132,081 | 0.09 |
| 5,000 | 247.3 | 35.6 | 40,000 | 132,081 | 0.46 |
| 10,000 | 399.4 | 21.9 | 80,000 | 132,081 | 0.93 |
| 20,000 | 758.0 | 26.4 | 160,000 | 132,081 | 1.85 |

## 9.8. Packed masked index

Table 12 compares the masked index with one membership bit per $\mathbb{Z}_p$ word against 32 bits per word, at the primary corpus's tag domain $N = 11{,}580$, in the reference implementation. Packing divides the index by exactly 32 and makes retrieval 19 to 27 times faster at $N_d = 10^4$ and $2 \times 10^4$, where both layouts fit in this machine's memory. At the full 405,184 records the packed index is 1.17 GB, against 37.5 GB unpacked (computed), and a retrieval takes 1.10 s and returns 101,296 B per provider. Every retrieval returned exactly the posting list.

**Table 12**
Masked index, unpacked against 32 bits per word, $N = 11{,}580$. Reference implementation, one thread; retrieval per provider.

| $N_d$ | Layout | Index (GB) | Retrieval (ms) | Downlink (B) |
|---|---|---|---|---|
| 10,000 | unpacked | 0.926 | 424.4 | 80,000 |
| 10,000 | 32 bits/word | 0.029 | 22.0 | 2,504 |
| 20,000 | unpacked | 1.853 | 696.2 | 160,000 |
| 20,000 | 32 bits/word | 0.058 | 26.0 | 5,000 |
| 100,000 | 32 bits/word | 0.289 | 234.6 | 25,000 |
| 405,184 | 32 bits/word | 1.173 | 1095.8 | 101,296 |

## 9.9. Multi-cloud deployment

With $n \in \{2, 3, 4\}$ separate provider processes and a client over TCP on one host (Table 13), provider compute is flat in $n$, at 34.82, 34.85 and 35.48 ms, so the client waits on the slowest provider, not on their sum. End-to-end latency rises from 44.1 to 52.5 ms, all of it transport of the $n$ responses; the client's combine,

verify and decrypt cost 1.37–1.63 ms. The transport figures exclude wide-area latency, which a deployment across regions adds unchanged.

**Table 13**
Single-host deployment over loopback, medians of 100 queries. Transport excludes wide-area latency.

| $n$ | End-to-end (ms) | Provider (ms) | Transport (ms) | Client (ms) |
|---|---|---|---|---|
| 2 | 44.12 | 34.82 | 8.30 | 1.37 |
| 3 | 45.40 | 34.85 | 9.09 | 1.52 |
| 4 | 52.53 | 35.48 | 15.48 | 1.63 |

## 9.10. Communication

At $N = 11{,}580$ the two-party construction sends a 266 B key to each provider, 532 B of uplink in total. The non-succinct $n$-party instantiation used for $n > 2$ sends 32 B to each of $n-1$ providers and 92,656 B to one, so total uplink is 92,688 B at $n = 2$, 92,720 B at $n = 3$ and 92,752 B at $n = 4$. Tolerating more collusion is therefore nearly free in this instantiation, but moving beyond two providers at all costs a factor of 174 in uplink over the succinct two-party tree. A succinct $(n-1)$-private instantiation would reduce the heavy key from $O(N \log p)$ to $O(2^{n/2}\sqrt{N}\lambda)$.

Downlink is 525,950 B per provider for an aggregate in the two-ciphertext layout and 132,081 B packed and compressed, for up to four templates in our configuration, and a fixed $\lceil N_d/32 \rceil$ words for a retrieval, which is 25,000 B at $N_d = 10^5$ packed (800,000 B unpacked), none depending on $|S|$. A range query and a flexible round each return one 132,081 B ciphertext.

## 9.11. Leakage evaluation

We mount a frequency and volume leakage-abuse attack in the style of Islam et al. [9] and Cash et al. [10] against our own index, twice: once against an unmasked replicated index, whose rows expose their Hamming weight and hence every posting-list length, and once against the masked index of Section 5.4, whose row statistics we compute with the real PRF over all $11{,}580 \times 405{,}184$ entries. The adversary's auxiliary knowledge is the tag-frequency distribution of a similar deployment, modelled by binomially thinning the true posting lengths to a sampled fraction of the records, so the attack is not handed the answer.

Aggregated over the whole tag universe (Fig. 3) the unmasked index yields 4.27% re-identification under perfect auxiliary knowledge, against 0.0086% for random guessing, a 500× lift but low in absolute terms. That aggregate is misleading, and Table 14 shows why: 11,523 of the 11,580 tags are one-minute time buckets of near-identical length, which volume leakage genuinely cannot separate. Every other tag class, meaning the device identifiers, the light and motion flags and all five sensor-band families, is re-identified *completely*, at 100%. These are precisely the tags that carry semantic meaning: which device, which sensor condition.

Against the masked index the same attack recovers 0.00% in every class and at every level of auxiliary knowledge. A more demanding comparison is with the standard padding defences rather than with no defence: padding each posting list to the next power of two, and volume-hiding padding of every row to the longest. The reference implementation provides the same attack against both (`bench/bench_leakage.py`); we have not yet run it on the corpora (Section 9.17). The masked row statistics are not distinguishable from uniform over $\mathbb{Z}_p$ by a 64-bin chi-square test ($\chi^2 = 72.27$ on 63 degrees of freedom, $p_{\chi^2} = 0.198$). This is the empirical counterpart of Theorem 1, evidence for it rather than a proof of it. We note that this experiment is deterministic given the corpus and the key seed, and reproduced to the digit on two different machines and microarchitectures. Section 9.15 repeats the attack on a twelve-site corpus with three times the tag domain and obtains the same per-class picture, so neither the 100% unmasked recovery nor the 0% masked recovery is an artefact of this corpus's three devices.

## 9.12. Setup, masking and ingest costs

Unpacked, the mask costs a factor of 64 in index storage over a plaintext bitmap; packed, a factor of 2. In the C++ prototype the unpacked masked index at $N_d = 10^5$ took 5.6 s to build, the fast-path column

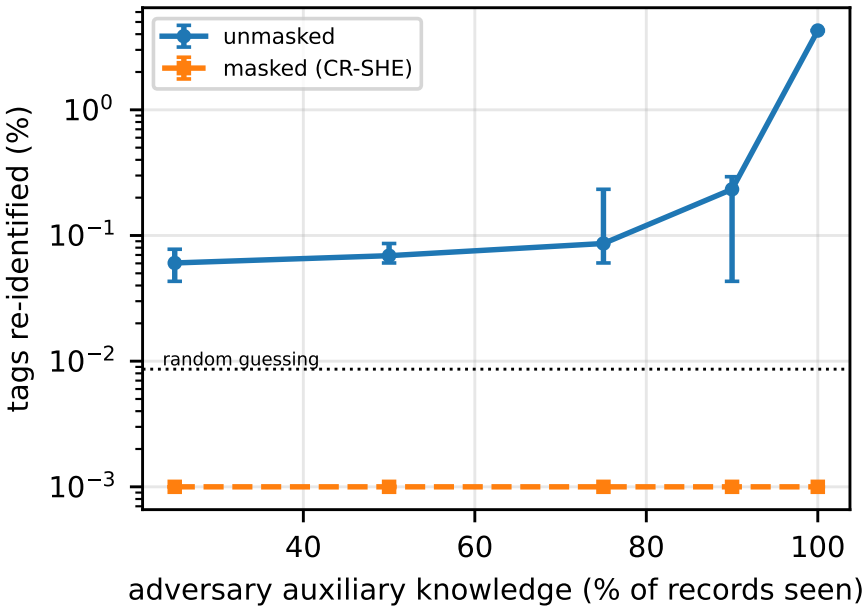



**Figure 3:** Tag re-identification against the unmasked and masked index, against the adversary's auxiliary knowledge. Table 14 breaks it down by class.

**Table 14**
Tag re-identification by metadata class, primary corpus, under perfect auxiliary knowledge.

| Tag class | Tags | Unmasked | Masked |
|---|---|---|---|
| Device identifier | 3 | 100% | 0% |
| Light flag | 2 | 100% | 0% |
| Motion flag | 2 | 100% | 0% |
| CO band | 10 | 100% | 0% |
| Humidity band | 10 | 100% | 0% |
| LPG band | 10 | 100% | 0% |
| Smoke band | 10 | 100% | 0% |
| Temperature band | 10 | 100% | 0% |
| One-minute window | 11,523 | 3.80% | 0% |
| Random guessing | n/a | 0.0086% | |

$A_f$ 1.2–12 ms per template, and a complete provider bundle for the full corpus (context, evaluation keys, $pk$ and eight columns) was 12 MB. Insertion into the hot buffer ran at 4.5–14.1 million records per second; a fast-path static-tier rebuild cost 14.7–14.9 ms per template, 58.0 $\mu$s per insertion at $B = 256$ and 0.91 $\mu$s at $B = 16{,}384$, while the buffer scan added 98.8–100.0% to a query. The range cube registered in Section 9.2 took 1.3 s (combined) and 9.8 s (per-corner) to build and encrypt in the reference implementation.

## 9.13. Core scaling and throughput

In the C++ prototype a single fast-path query does not speed up with cores (34.3, 34.4 and 38.4 ms at 1, 8 and 64 threads), because the homomorphic contraction dominates, but throughput does: 29.1 queries/s with one client, 110.2 with four, and a plateau of 121.6–127.7 from sixteen to sixty-four clients, with median latency rising from 34.4 to 516 ms.

## 9.14. Verification, and what it does not cover

Packed into the second slot row, the MAC costs no extra products, rotations or downlink. Table 15 measures four provider deviations over 200 trials each in the reference implementation. A single MAC key catches every inconsistent response and none of the consistent substitutions, uniform or scaled, as Section 5.8 predicts; the C++ prototype gave the same 200/200 and 0/200. Per-row keys catch all of them except a perturbation of the hidden row itself: perturbing one random row was caught in 174 trials, and in the other 26 that row's aggregate was zero, so the answer stayed correct. The range path's per-corner keys behave the same way (Lemma 2); its tests include tampering with a real and with a dummy corner. On authorisation, a client sending raw additive shares of basis vectors recovered 512 of 512 rows of a synthetic index with 512 queries, undetected, since each share is uniform: what any client with query access can do, and what a per-user authorisation mechanism must prevent.

**Table 15**
MAC detection over 200 trials per cell: detected / undetected and wrong / undetected and still correct. $N = 11{,}580$, reference implementation.

| Provider deviation | Single $\delta$ | Per-row $\delta_x$ |
|---|---|---|
| Constant on value row | 200 / 0 / 0 | 200 / 0 / 0 |
| $e_j + d$, $d$ uniform | 0 / 200 / 0 | 200 / 0 / 0 |
| $k\,e_j$ | 0 / 200 / 0 | 200 / 0 / 0 |
| $e_j + c\,u_x$, random $x$ | 0 / 168 / 32 | 174 / 0 / 26 |

## 9.15. Second corpus: twelve sites and a larger tag domain

The primary corpus has three devices, so its tag universe is dominated by time buckets. The C++ prototype therefore repeated every experiment above on the Beijing corpus, whose twelve sites give three times the tag domain ($N = 35{,}162$, posting lengths median 12, mean 131.5, maximum 102,344). Missing readings get their own `:na` tag rather than falling into the lowest band, since CO is missing in 4.92% of records, and temperature is shifted by $+50\,^{\circ}\mathrm{C}$ so that it stays non-negative. Every correctness check passed unchanged. The fast path cost 40.2–41.2 ms from $N_d = 5 \times 10^3$ to $4.05 \times 10^5$, 5% more than on the primary corpus for 3.04× the tags. The unmasked index again gave up every one of the ten non-temporal tag classes at 100%, including the twelve sites and seventeen wind directions, and the hour buckets at 0.31%; the masked index gave up 0.00% in every class, its row sums indistinguishable from uniform ($\chi^2 = 62.38$ on 63 degrees of freedom, $p_{\chi^2} = 0.4986$). The unpacked masked index needed 28.13 GB at $N_d = 10^5$ and would need 118.2 GB at full scale, which the packed layout of Section 5.4 divides by 32.

## 9.16. Discussion

*Ranges are where the design pays.* A registered range aggregate costs the same for a one-minute window as for the whole timeline, returns one ciphertext, and sends eight keys of 345 B; the single-tag path would need one query per cell, the flexible path one retrieval per tag, and a single server an encrypted query as long as the cube. On single-keyword aggregates the trust split saves only 1.8–2.9× in server work over a single server (Section 9.6); on ranges the client-side gap reaches four orders of magnitude in uplink, which is the regime a thin IoT client is in.

*Cube size is the provisioning knob.* Range cost grows linearly in the number of cells, 1.36 s per provider at $10^6$ cells in our unoptimised implementation. A deployment registers the cubes its analysts need, as it registers templates, and the cell count, not the record count, bounds the per-query cost. The per-corner layout multiplies storage and work by $2^d$ in exchange for verifying every corner; the combined layout does not.

*Retrieval is now bounded by time, not storage.* Packing brings the masked index from 37.5 to 1.17 GB per provider at the full primary corpus, so the flexible path, which serves arbitrary selections in two rounds, fits on commodity hosts.

## 9.17. Limitations

*Evaluation.* Range aggregation and the packed index were measured in a Python reference implementation on a synthetic cube shaped like the primary corpus, not in the C++ prototype on the real corpora; their

absolute latencies are those of an unoptimised driver. Providers in the C++ deployment ran on one host, so no wide-area latency was measured.

*Expressiveness.* Range queries are limited to registered cubes and templates, and the cube's total must stay below $p$; a sum of squares over the whole corpus would need a larger plaintext modulus. Non-rectangular predicates and conjunctions across cubes use the two-round flexible path.

*Adversary.* Providers are semi-honest. Per-cell and per-row MAC keys detect deviations except those concentrated on the hidden point; the combined range layout and the flexible path have only a global MAC; nothing guarantees output delivery. Per-user authorisation remains a requirement the prototype does not meet (Section 9.14).

*Collusion beyond two providers.* For $n > 2$ the prototype uses a non-succinct sharing whose heavy key is $8N + 16$ bytes; a range query needs $2^d$ such keys over the cube's $M$ cells. Our key-size claims are for $n = 2$.

*Prior art.* The full texts of the closest works on private aggregation and range queries could not be read from our environment (Section 2); our statements about them rest on abstracts and search summaries.

## 10. Conclusion

IoT analytics are range aggregates over time, place and value, and the data behind them increasingly sit encrypted across several clouds. CR-SHE answers such aggregates in one round, over data encrypted under one owner's key, hiding the range, the matched records, their number and the result from any coalition of up to $n - 1$ providers that never talk to one another. It rests on three pieces: a proof that this setting needs a homomorphic layer at all, oblivious homomorphic aggregation, in which each provider contracts its own share of the selection against encrypted columns and returns one ciphertext, and encrypted summed-area tables, which turn any box of a registered data cube into $2^d$ point selections whose signs and dummy padding hide the box entirely. A packed masked index and a two-round flexible path cover the queries a cube does not. Measured costs depend on the cube's size, not on the range or the record count, and the client sends kilobytes where a single server would need tens of megabytes. Future work will port range aggregation to the native prototype, evaluate it across cloud regions and on gateway hardware, and pursue succinct multi-party keys and verification against providers that deviate on purpose.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

## Data availability

Data will be made available on request.

## Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic), an AI assistant, in order to convert the manuscript between journal templates, edit and restructure the text, write the plotting script that draws Fig. 1, develop the reference implementation with its tests and benchmark scripts, search the literature for prior work on private range aggregation, and draft parts of the text and analysis, including the range construction, Lemma 2 and its results in Sections 5.7 and 9.2, and Sections 5.2, 5.10, 5.6, 9.6, 9.7 and 9.8. No image was generated or altered by a generative AI tool. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

## References


[1] L. Atzori, A. Iera, G. Morabito, The Internet of Things: A survey, Computer Networks 54 (2010) 2787–2805.

[2] D. Petcu, Multi-cloud: Expectations and current approaches, in: Proc. Int. Workshop on Multi-Cloud Applications and Federated Clouds (MultiCloud), 2013, pp. 1–6.

[3] N. Grozev, R. Buyya, Inter-cloud architectures and application brokering: Taxonomy and survey, Software: Practice and Experience 44 (2014) 369–390.

[4] D. X. Song, D. Wagner, A. Perrig, Practical techniques for searches on encrypted data, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2000, pp. 44–55.

[5] R. Curtmola, J. Garay, S. Kamara, R. Ostrovsky, Searchable symmetric encryption: Improved definitions and efficient constructions, in: Proc. 13th ACM Conference on Computer and Communications Security (CCS), 2006, pp. 79–88.

[6] M. Islam, R. Palit, A keyword based searching and sharing scheme on the encrypted cloud data, in: 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), IEEE, 2023, pp. 1–6.

[7] J. Fan, F. Vercauteren, Somewhat practical fully homomorphic encryption, Cryptology ePrint Archive (2012).

[8] J. H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, Cryptology ePrint Archive, Paper 2016/421, 2016. URL: https://eprint.iacr.org/2016/421.

[9] M. S. Islam, M. Kuzu, M. Kantarcioglu, Access pattern disclosure on searchable encryption: Ramification, attack and mitigation, in: Proc. Network and Distributed System Security Symposium (NDSS), 2012.

[10] D. Cash, P. Grubbs, J. Perry, T. Ristenpart, Leakage-abuse attacks against searchable encryption, in: Proc. 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS), 2015, pp. 668–679.

[11] P. Grubbs, K. Sekniqi, V. Bindschaedler, M. Naveed, T. Ristenpart, Leakage-abuse attacks against order-revealing encryption, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2017, pp. 655–672.

[12] F. Wang, C. Yun, S. Goldwasser, V. Vaikuntanathan, M. Zaharia, Splinter: Practical private queries on public data, in: Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2017, pp. 299–313.

[13] E. Dauterman, E. Feng, E. Luo, R. A. Popa, I. Stoica, DORY: An encrypted search system with distributed trust, in: Proc. 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2020, pp. 1101–1119.

[14] E. Dauterman, M. Rathee, R. A. Popa, I. Stoica, Waldo: A private time-series database from function secret sharing, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2022.

[15] N. Gilboa, Y. Ishai, Distributed point functions and their applications, in: Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2014, pp. 640–658.

[16] E. Boyle, N. Gilboa, Y. Ishai, Function secret sharing, in: Advances in Cryptology – EUROCRYPT 2015, volume 9057 of *LNCS*, Springer, 2015, pp. 337–367.

[17] E. Boyle, N. Gilboa, Y. Ishai, Function secret sharing: Improvements and extensions, in: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security, 2016, pp. 1292–1303.

[18] S. Eskandarian, H. Corrigan-Gibbs, M. Zaharia, D. Boneh, Express: Lowering the cost of metadata-hiding communication with cryptographic privacy, in: Proc. 30th USENIX Security Symposium, 2021.

[19] H. Corrigan-Gibbs, D. Boneh, Prio: Private, robust, and scalable computation of aggregate statistics, in: Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2017, pp. 259–282.

[20] D. Boneh, E. Boyle, H. Corrigan-Gibbs, N. Gilboa, Y. Ishai, Lightweight techniques for private heavy hitters, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2021, pp. 762–776.

[21] S. Servan-Schreiber, S. Beyzerov, E. Yablon, H. Park, Private access control for function secret sharing, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2023.

[22] L. de Castro, A. Polychroniadou, Lightweight, maliciously secure verifiable function secret sharing, in: Advances in Cryptology – EUROCRYPT 2022, volume 13275 of *LNCS*, Springer, 2022.

[23] B. Chor, E. Kushilevitz, O. Goldreich, M. Sudan, Private information retrieval, Journal of the ACM 45 (1998) 965–981.

[24] A. Beimel, Y. Ishai, T. Malkin, Reducing the servers' computation in private information retrieval: PIR with preprocessing, in: Advances in Cryptology – CRYPTO 2000, volume 1880 of *LNCS*, Springer, 2000, pp. 55–73.

[25] C.-T. Ho, R. Agrawal, N. Megiddo, R. Srikant, Range queries in OLAP data cubes, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 1997, pp. 73–88.

[26] F. C. Crow, Summed-area tables for texture mapping, in: Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH), 1984, pp. 207–212.

[27] I. Demertzis, et al., Practical private range search in depth, ACM Transactions on Database Systems (2018). Full text not read; cited from its abstract.

[28] Z. Espiritu, et al., Time- and space-efficient aggregate range queries over encrypted databases, Proceedings on Privacy Enhancing Technologies 2022 (2022). Full text not read; cited from a search summary.

[29] X. Liu, et al., HADES: Range-filtered private aggregation on public data, Proceedings of the VLDB Endowment 18 (2025) 2159–. Full text not read; cited from a search summary.

[30] J. Hayata, et al., On private information retrieval supporting range queries, International Journal of Information Security (2020). Full text not read; cited from its abstract.

[31] S. M. Hafiz, et al., Private aggregate queries to untrusted databases, in: Network and Distributed System Security Symposium (NDSS), 2024. Full text not read; cited from its abstract.

[32] H. Corrigan-Gibbs, D. Kogan, Private information retrieval with sublinear online time, in: Advances in Cryptology – EUROCRYPT 2020, LNCS, Springer, 2020.

[33] H. Corrigan-Gibbs, A. Henzinger, D. Kogan, Single-server private information retrieval with sublinear amortized time, in: Advances in Cryptology – EUROCRYPT 2022, LNCS, Springer, 2022.
[34] M. Zhou, A. Park, W. Zheng, E. Shi, Piano: Extremely simple, single-server PIR with sublinear server computation, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2024.
[35] S. Angel, H. Chen, K. Laine, S. Setty, PIR with compressed queries and amortized query processing, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2018.
[36] M. H. Mughees, H. Chen, L. Ren, OnionPIR: Response efficient single-server PIR, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2021.
[37] S. J. Menon, D. J. Wu, Spiral: Fast, high-rate single-server PIR via FHE composition, in: Proc. IEEE Symposium on Security and Privacy (S&P), 2022.
[38] A. Davidson, G. Pestana, S. Celi, FrodoPIR: Simple, scalable, single-server private information retrieval, Proceedings on Privacy Enhancing Technologies 2023 (2023).
[39] E. Kushilevitz, R. Ostrovsky, Replication is not needed: Single database, computationally-private information retrieval, in: Proceedings of the 38th Annual Symposium on Foundations of Computer Science (FOCS), IEEE, 1997, pp. 364–373.
[40] O. Goldreich, R. Ostrovsky, Software protection and simulation on oblivious rams, Journal of the ACM (JACM) 43 (1996) 431–473.
[41] S. Kamara, T. Moataz, Computationally volume-hiding structured encryption, in: Advances in Cryptology – EUROCRYPT 2019, LNCS, Springer, 2019.
[42] S. Patel, G. Persiano, K. Yeo, M. Yung, Mitigating leakage in secure cloud-hosted data structures: Volume-hiding for multi-maps via hashing, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2019.
[43] W. Xu, J. Zhang, Y. Yuan, Z. Li, Privacy-preserving multi-cloud based dynamic symmetric searchable encryption, in: 2021 2nd International Conference on Computer Communication and Network Security (CCNS), IEEE, 2021, pp. 176–181.
[44] W. Xu, J. Zhang, Y. Yuan, X. Wang, Symmetric searchable encryption with supporting search pattern and access pattern protection in multi-cloud, Concurrency and Computation: Practice and Experience 35 (2023) e7651.
[45] X. Zhang, C. Huang, Y. Su, J. Qin, Secure, dynamic, and efficient keyword search with flexible merging for cloud storage, IEEE Transactions on Services Computing 17 (2024) 2822–2835.
[46] N. Yang, C. Tang, Q. Zhou, D. He, Dynamic consensus committee-based for secure data sharing with authorized multi-receiver searchable encryption, IEEE Transactions on Information Forensics and Security 18 (2023) 5186–5199.
[47] R. Du, H. Jiang, M. Li, Lightweight searchable encryption with small clients on edge cloud, in: 2022 17th Asia Joint Conference on Information Security (AsiaJCIS), 2022, pp. 41–48. doi:10.1109/AsiaJCIS57030.2022.00017.
[48] S. Halder, T. Newe, Enabling secure time-series data sharing via homomorphic encryption in cloud-assisted iiot, Future Generation Computer Systems 133 (2022).
[49] A. Kamble, M. M. Jiet, C. Puri, Homomorphic encryption and its applications in multi-cloud security, in: 2024 International Conference on Inventive Computation Technologies (ICICT), IEEE, 2024, pp. 1493–1499.
[50] L. Burkhalter, A. Hithnawi, A. Viand, H. Shafagh, S. Ratnasamy, TimeCrypt: Encrypted data stream processing at scale with cryptographic access control, in: Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2020, pp. 835–850.
[51] L. Burkhalter, N. Küchler, A. Viand, H. Shafagh, A. Hithnawi, Zeph: Cryptographic enforcement of end-to-end data privacy, in: Proc. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2021, pp. 387–404.
[52] C. Xu, N. Wang, L. Zhu, K. Sharif, C. Zhang, Achieving searchable and privacy-preserving data sharing for cloud-assisted e-healthcare system, IEEE Internet of Things Journal 6 (2019) 8345–8356.
[53] D. Wu, C. Nie, Z. Yang, P. Zhang, R. Wang, Distributed secure data sharing method for internet of vehicles based on homomorphic hash tree, IEEE Transactions on Vehicular Technology 73 (2024) 17572–17585.
[54] A. K. Verma, P. Patel, V. S. Bhatia, A. Rastogi, A. Prasad, P. Ranjan, Secure federated multi-party computation with homomorphic encryption and blockchain for preserving privacy in cloud storage of medical records, in: 2025 2nd International Conference on Intelligent Algorithms for Computational Intelligence Systems (IACIS), IEEE, 2025, pp. 1–6.
[55] N. Bharot, N. Mehta, J. G. Breslin, P. Verma, Cloudlock: secure data sharing using a hybrid cryptosystem in multi-cloud data storage, Cluster Computing 28 (2025) 464.
[56] S. Muthuvel, S. Priya, K. Kumar, Design of a multi-layered privacy-preserving architecture for secure medical data exchange in cloud environments, Scientific Reports 16 (2026).
[57] L. Wang, Z. Yang, X. Song, Shamc: A secure and highly available database system in multi-cloud environment, Future Generation Computer Systems 105 (2020) 873–883.
[58] D. Horkos, L. Perret, SoK: On cryptography for multi-cloud storage, Cryptology ePrint Archive, Paper 2026/207, 2026. URL: https://eprint.iacr.org/2026/207.
[59] R. Poddar, T. Boelter, R. A. Popa, Arx: An encrypted database using semantically secure encryption, Proceedings of the VLDB Endowment 12 (2019).
[60] N. Crooks, M. Burke, E. Cecchetti, S. Harel, R. Agarwal, L. Alvisi, Obladi: Oblivious serializable transactions in the cloud, in: Proc. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018.
[61] E. Dauterman, V. Fang, I. Demertzis, N. Crooks, R. A. Popa, Snoopy: Surpassing the scalability bottleneck in oblivious storage, in: Proc. 28th ACM Symposium on Operating Systems Principles (SOSP), 2021.
[62] O. Goldreich, S. Goldwasser, S. Micali, How to construct random functions, in: providing sound foundations for cryptography: On the Work of Shafi Goldwasser and Silvio Micali, 2019, pp. 241–264.

[63] G. Di Crescenzo, T. Malkin, R. Ostrovsky, Single database private information retrieval implies oblivious transfer, in: Advances in Cryptology – EUROCRYPT 2000, volume 1807 of *Lecture Notes in Computer Science*, Springer, 2000, pp. 122–138.

[64] S. Halevi, V. Shoup, Algorithms in HElib, in: Advances in Cryptology – CRYPTO 2014, volume 8616 of *Lecture Notes in Computer Science*, Springer, 2014, pp. 554–571.

[65] A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, et al., OpenFHE: Open-source fully homomorphic encryption library, in: Proc. 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC), 2022, pp. 53–63.

[66] G. A. Stafford, [dataset] environmental sensor telemetry data, Kaggle Datasets, 2020. Version 1.0. https://www.kaggle.com/datasets/garystafford/environmental-sensor-data-132k.

[67] S. Zhang, B. Guo, A. Dong, J. He, Z. Xu, S. X. Chen, Cautionary tales on air-quality improvement in Beijing, Proceedings of the Royal Society A 473 (2017) 20170457. Dataset: UCI Machine Learning Repository, Beijing Multi-Site Air-Quality Data, https://archive.ics.uci.edu/dataset/501.

[68] S. Chen, [dataset] Beijing multi-site air quality, UCI Machine Learning Repository, 2019. doi:10.24432/C5RK5G, https://doi.org/10.24432/C5RK5G.